instituto superior de engenharia de lisboa

Real-Time License Plate Recognition Using Smart Edge Devices for Surveillance

SEBASTIÃO PÓVOA FERREIRA
(Licenciado)

Trabalho de Projeto para obtenção do Grau de Mestre em
Engenharia Informática e Multimédia

ORIENTADOR
Doutor Mário Pereira Véstias

Setembro 2026

instituto superior de engenharia de lisboa

Real-Time License Plate Recognition Using Smart Edge Devices for Surveillance

SEBASTIÃO PÓVOA FERREIRA

(Licenciado)

Trabalho de Projeto para obtenção do Grau de Mestre em

Engenharia Informática e Multimédia

ORIENTADOR

Doutor Mário Pereira Véstias, ISEL

JÚRI

Presidente

Doutor Pedro Miguel Torres Mendes Jorge, ISEL

Vogais

Doutor Mário Pereira Véstias, ISEL

Doutor Rui António Policarpo Duarte, ISEL

Setembro 2026

Acknowledgments

I would like to begin by thanking my supervisor, Doutor Mário Pereira Véstias, for his guidance throughout the development of this thesis. His availability for discussion, his constructive feedback at every stage, and his help through difficult technical roadblocks of this work truly made all the difference. I am also grateful for the opportunity to present and develop this research through two academic conferences during the course of this project.

My gratitude also extends to Instituto Superior de Engenharia de Lisboa for providing the knowledge and resources that allowed me to successfully complete this work.

I am deeply grateful to my fellow students and colleagues, thank you for your camaraderie and support throughout these years. Your help on group projects, and the shared ideas even on individual ones, truly gave me everything i needed to reach this point in my academic career.

To my friends, who always gave me the motivation and encouragement to keep going when progress on this work felt slow, thank you for your patience and understanding.

Last but not least, to my family, thank you for your unconditional support throughout not only this thesis but my entire academic path. You always kept my head held high and never let me give up, even when i wanted to. If it weren't for you, i wouldn't be here today, able to write these very words.

Statement of integrity

I declare that this dissertation is the result of my personal and independent research. Its content is original, and all sources listed in the bibliographic references were consulted and are duly mentioned in the text. I further declare that all scientific and technical references relevant to the development of the work are duly cited and included in the bibliographic references.

The author

Lisbon, september - 26, 2026

Real-Time License Plate Recognition Using Smart Edge Devices for Surveillance

Resumo

O roubo de veículos e as infrações rodoviárias representam desafios significativos para a segurança e gestão urbana. Como os métodos tradicionais de identificação visual são lentos e suscetíveis a falhas, os avanços em inteligência artificial impulsionaram o desenvolvimento de sistemas de Reconhecimento Automático de Matrículas (ALPR) em tempo real. Esta dissertação apresenta um sistema baseado em *deep learning* para automatizar a deteção e transcrição de matrículas europeias.

Grande parte das soluções ALPR atuais depende da *cloud* ou de GPUs de servidor, enfrentando limitações severas de latência, memória e energia quando operam localmente (*edge*). Para resolver este problema, o projeto investiga, otimiza e implementa uma *pipeline* ALPR de duas fases, especificamente adaptada a *hardware edge*.

Após avaliação, selecionaram-se um modelo YOLOv8n (deteção) e um motor PaddleOCR (reconhecimento) para implementação no microcomputador Khadas VIM3. A carga computacional é distribuída eficazmente pela arquitetura do dispositivo: a extração de características é acelerada na Unidade de Processamento Neural (NPU) de 5.0 TOPS em precisão FP16, enquanto o pós-processamento e a descodificação são delegados para o CPU ARM.

A avaliação experimental demonstra que a deteção atinge um mAP@50 de 82,41% e o reconhecimento obtém 89,65% de correspondência exata de sequência, com 98,06% de similaridade de carateres. A *pipeline* completa opera a 5,95 FPS (167,96 ms de latência). Estes resultados validam o objetivo principal do projeto: comprovar a viabilidade de sistemas ALPR autónomos, precisos e em tempo real, executados de forma totalmente local (*on-device*) com aceleradores *edge* económicos.

Palavras-chave: Reconhecimento Automático de Matrículas, Edge Computing, Deep Learning, Khadas VIM3, Deteção de Objetos, OCR.

Real-Time License Plate Recognition Using Smart Edge Devices for Surveillance

Abstract

Vehicle theft, illegal parking, and traffic violations remain significant challenges for law enforcement agencies and urban management authorities. Traditional vehicle identification methods based on manual visual inspection are time-consuming and prone to human error. Advances in artificial intelligence and computer vision have enabled the development of Automatic License Plate Recognition (ALPR) systems capable of identifying vehicles in real time. This thesis presents a deep-learning-based license plate recognition system designed to automate the detection and transcription of European license plates.

While numerous ALPR solutions have been proposed, many rely on cloud connectivity or heavy server-grade GPUs, facing severe limitations when deployed under real-world edge constraints such as latency, memory, and energy efficiency. To address these challenges, this project investigates, optimizes, and implements a lightweight, two-stage ALPR pipeline tailored specifically for resource-constrained edge hardware.

Following baseline benchmarking, a YOLOv8n detection model and a PaddleOCR recognition engine were selected and optimized for deployment on a Khadas VIM3 single-board computer. The system effectively distributes the computational workload across the device's heterogeneous architecture, offloading feature extraction to the 5.0 TOPS Neural Processing Unit (NPU) at FP16 precision while delegating data-dependent post-processing and sequence decoding to the ARM CPU cluster.

Experimental evaluation on the target hardware demonstrates that the detection stage achieves a mAP@50 of 82.41%, while the recognition stage yields an exact sequence match rate of 89.65% and a character similarity of 98.06%. The combined end-to-end pipeline sustains a throughput of 5.95 FPS (167.96 ms latency). These results successfully validate the project's primary objective: establishing that high-accuracy, real-time license plate recognition can be achieved entirely on-device using low-cost edge accelerators.

Keywords: Automatic License Plate Recognition, Edge Computing, Deep Learning, Khadas VIM3, Object Detection, Optical Character Recognition.

Contents

List of Figures vii

List of Tables viii

Acronyms ix

1 Introduction 1
- 1.1 Problem Statement . . . 1
- 1.2 Objectives . . . 2
- 1.3 Research Outcomes . . . 2
- 1.4 Document Organization . . . 3

2 State of the Art in License Plate Recognition 4
- 2.1 Background . . . 4
  - 2.1.1 Fundamentals of License Plate Recognition . . . 4
  - 2.1.2 Deep Learning . . . 7
  - 2.1.3 Deep learning for LPR . . . 8
  - 2.1.4 OCR using CNN and Transformer-based Models . . . 9
  - 2.1.5 Importance of Datasets in Deep Learning . . . 9
  - 2.1.6 Edge Computing . . . 10
  - 2.1.7 Applications of LPR . . . 11
- 2.2 Related Work in License Plate Recognition . . . 12
  - 2.2.1 Traditional LPR Systems . . . 12
  - 2.2.2 Relevant LPR Architectures . . . 13
  - 2.2.3 Edge-Based LPR Systems . . . 15
  - 2.2.4 Performance Benchmarks . . . 16
  - 2.2.5 Commercial and Open-Source Industry Solutions . . . 17
- 2.3 Discussion and Research Gap . . . 17

3 Methodology and Benchmarking 19
- 3.1 Proposed Architecture for LPR . . . 20
- 3.2 Dataset Selection . . . 22
- 3.3 Evaluation Metrics . . . 22
- 3.4 Baseline Model Selection . . . 23

3.4.1 Detection Model Benchmarking . . . 24
3.4.2 Recognition Model Benchmarking . . . 26
3.5 Summary . . . 27

4 Model Optimization for Khadas VIM3 28
4.1 Target Hardware Description . . . 28
4.2 Model Conversion and Compilation . . . 29
4.2.1 Acuity Toolkit Optimization Methodology . . . 29
4.2.2 Detector Reselection: From YOLOv26n to YOLOv8n . . . 30
4.2.3 YOLOv8n Conversion Pipeline . . . 31
4.2.4 PaddleOCR Conversion Pipeline . . . 32
4.2.5 PaddleOCR NPU Compilation Failure and CPU Fallback . . . 33
4.2.6 PaddleOCR Optimization Ablation . . . 34
4.3 End-to-End Pipeline Design . . . 35
4.3.1 NPU: Hardware-Accelerated Feature Extraction . . . 36
4.3.2 CPU: Control Flow and Post-Processing . . . 36
4.3.3 Combined Pipeline Flow . . . 37
4.4 Summary . . . 37

5 Experimental Evaluation and Results 39
5.1 Experimental Setup . . . 39
5.2 System Performance . . . 39
5.2.1 Object Detection Performance . . . 39
5.2.2 Character Recognition Performance . . . 41
5.2.3 Full System Performance . . . 41
5.3 Qualitative Results . . . 42
5.4 Comparison to Related Work . . . 45
5.5 Summary . . . 46

6 Conclusion and Future Work 47
6.1 Conclusion . . . 47
6.2 Future Work . . . 48

Bibliography 50

List of Figures

2.1 Overview of the LPR System Pipeline 5

3.1 Proposed Two-Stage Architecture for the Edge-Based LPR Pipeline . . 20

4.1 Hardware and Software Partitioning Across Khadas VIM3 Heterogeneous Compute Domains 35

5.1 Example of a captured vehicle frame showing standard European plate proportions. Recognized text: "AA01AA"(ground truth: "AA 01 AA"). 43

5.2 Example of a close-up rear capture of a license plate. Recognized text: "6900MA"(ground truth: "69 00 MA") 43

5.3 Example of a synthetic vehicle image showing a rear-mounted license plate. Recognized text: "N88"(ground truth: "NOH E 8843") 44

List of Tables

2.1 Summary of Baseline Performance Metrics Reported in Literature . . . 14

3.1 Inference Speed Comparison of Detection Models 24
3.2 Detailed Precision and Latency Benchmarks for YOLO Architectures . 25
3.3 Baseline Performance of Out-of-the-Box Recognition Engines 26

4.1 Impact of Pruning and Quantization on YOLOv26n 30
4.2 Impact of Pruning and Quantization on YOLOv8n 32
4.3 Ablation Study of PaddleOCR Optimization Techniques 34

5.1 YOLOv8n Object Detection Performance Metrics 40
5.2 Latency Comparison Between CPU and NPU Execution 40
5.3 PaddleOCR Recognition Performance Metrics 41
5.4 End-to-End Pipeline Latency Breakdown 42
5.5 Comparative Performance Evaluation Against Baselines and Literature 45

Acronyms

ALPR	Automatic License Plate Recognition 1, 4, 11, 12, 17, 47, 48
ANN	Artificial Neural Networks 7
API	Application Programming Interface 19, 29, 37, 39
ASCII	American Standard Code for Information Interchange 6
ASIC	Application-specific integrated circuits 10
BiLSTM	Bidirectional Long Short-Term Memory 9, 13
CCPD	Chinese City Parking Dataset 26, 27
CER	Character Error Rate 23, 46, 47
CNN	Convolutional Neural Networks 7, 8, 9, 13, 14, 34
COCO	Common Objects in Context 24
CPU	Central Processing Unit 2, 10, 17, 19, 20, 27, 28, 34, 35, 36, 37, 38, 39, 40, 41, 42, 45, 46, 47, 48
CRNN	Convolutional Recurrent Neural Network 13, 14, 18
CTC	Connectionist Temporal Classification 9, 14, 18, 21, 23, 34, 36, 37, 38, 39, 42, 44, 46
ETC	Electronic Toll Collection 11
FP16	16-bit Floating Point 3, 29, 30, 32, 35, 36, 37, 38, 40, 41, 47
FP32	32-bit Floating Point 29, 30, 32
FPGA	Field Programmable Gate Array 10, 15
FPS	Frames Per Second 2, 15, 16, 23, 42, 45, 46, 47, 48
GPU	Graphics Processing Unit 3, 8, 10, 14, 15, 18, 19, 23, 27, 28, 30, 31, 37, 40, 41, 45, 46
GRU	Gated Recurrent Unit 8
HTR	Handwritten Text Recognition 46
INT8	8-bit Integer 28, 29, 32
IoT	Internet of Things 11

IoU Intersection over Union 21

ITS Intelligent Transportation Systems 1, 11

KSNN Khadas Sensor Neural Network 19, 29, 31, 32, 34, 36, 37, 39

LPR License Plate Recognition 1, 2, 3, 4, 7, 8, 9, 10, 11, 12, 15, 16, 19, 27

LSTM Long Short-Term Memory 8, 13

ML Machine Learning 7

NMS Non-Maximum Suppression 13, 21, 23, 36, 39, 40, 42

NPU Neural Processing Unit 3, 19, 20, 24, 27, 28, 29, 30, 31, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 45, 46, 47, 48

OCR Optical Character Recognition 1, 2, 4, 6, 7, 8, 9, 13, 14, 21, 26, 27, 45, 46, 48

ONNX Open Neural Network Exchange 19, 21, 30, 31, 32, 33, 34, 36, 38, 39, 40, 41, 46, 48

R-CNN Region-based Convolutional Neural Network 2, 8, 15, 17, 20, 24, 25, 27, 45, 46

RFID Radio Frequency Identification 11

RNN Recurrent Neural Network 8, 9

RoI Region of Interest 6, 20, 21, 23, 36, 37, 38, 39, 42, 44, 46

SDK Software Development Kit 15, 30

SoC System-on-Chip 19, 28, 29, 37

SSD Single Shot MultiBox Detector 8, 20, 24, 25, 27

SVTR Single Visual model for Text Recognition 34

TOPS Tera Operations Per Second 19, 28, 29, 37, 39, 40, 41, 46, 47

TPU Tensor Processing Unit 10

ViT	Vision Transformer 9
YOLO	You Only Look Once 2, 8, 9, 13, 15, 17, 20, 24, 25, 27, 48

1 INTRODUCTION

In recent years, the number of vehicles continues to grow, and with it the need for systems capable of quick and precise identification. This necessity originates not only from the logistical challenges of managing denser traffic flows but also from the role of aiding law enforcements in the apprehension of vehicles involved in criminal activities or traffic violations [2]. As a direct response to these threats, Intelligent Transportation Systems (ITS), such as the Automatic License Plate Recognition (ALPR) system, have evolved significantly, integrating advanced technologies, such as Deep Learning and Computer Vision.

However, the architectural paradigm of these systems is currently undergoing a decisive shift. Traditionally, intelligent systems relied on transmitting data to centralized cloud servers, which offer virtually unlimited storage and computational resources. Although effective for offline analysis, this model is increasingly proving inadequate for real-time applications due to bandwidth limitations and privacy concerns. Consequently, there is a growing industry trend towards Edge Computing, where computational tasks are moved closer to the edge near the physical system [20]. By processing data locally on the device, edge solutions improve operational autonomy and immediate response times, which are essential to identify fleeing vehicles in high-speed.

1.1 Problem Statement

The central problem addressed by this thesis is the difficulty of deploying reliable License Plate Recognition (LPR) capabilities on resource-constrained edge devices. While state-of-the-art Deep Learning models achieve high accuracy on server-grade hardware, migrating these workloads to the edge introduces three critical constraints that current solutions often fail to balance: latency, computational resources, and energy efficiency [27].

Traffic surveillance requires real-time processing to capture fast-moving vehicles, but benchmarks reveal that traditional two-stage detectors and general-purpose Optical Character Recognition (OCR) engines are computationally too heavy, often

resulting in frame rates insufficient for dynamic traffic environments. Furthermore, low-cost edge devices are limited in memory and processing power compared to server-grade infrastructures, meaning that standard models are often too large to run efficiently. Finally, existing commercial solutions often rely on expensive high-end embedded hardware or constant cloud connectivity, creating a lack of unified software pipelines capable of sustaining real-time performance on resource-constrained hardware.

1.2 Objectives

The primary objective of this project is to design, evaluate, and implement a lightweight, deep learning-based LPR pipeline optimized for real-time inference on edge devices.

To achieve this, the project will first analyse existing research on object detection and character recognition to identify architectures suitable for embedded deployment, contrasting traditional image processing with modern Deep Learning approaches. Subsequently, a quantitative evaluation of detection models and recognition engines will be conducted to establish a baseline for speed and accuracy on non-accelerated hardware. Based on these benchmarks, the optimal technological stack will be selected, optimized, and integrated to meet a defined set of performance targets for the pipeline: a pipeline inference latency of no more than 200 ms per frame (at least 5 Frames Per Second (FPS)), a detection accuracy of at least 75% mAP@50, and a recognition accuracy of at least 75% exact-match on the character sequence, ensuring the system remains viable for fixed-camera traffic surveillance despite the resource constraints of embedded edge hardware.

1.3 Research Outcomes

The work developed in this dissertation was presented at two peer-reviewed conferences and published in their proceedings:

- **YEF-ECE conference** [5]: established the project's preliminary baseline. Benchmarking on unaccelerated Central Processing Unit (CPU) hardware showed that two-stage detection (Faster Region-based Convolutional Neural Network (R-CNN), 3377.82 ms) is unsuitable for real-time edge deployment, and identified single-stage You Only Look Once (YOLO) detectors paired with lightweight, segmentation-free OCR engines as the most promising direction going forward.

- **INForum conference** [6]: detailed the project's hardware-optimization experiments. Demonstrated that 16-bit Floating Point (FP16) precision quantization on an NVIDIA Jetson Orin Nano preserved detection accuracy above 99% while achieving real-time throughput, establishing FP16 as a viable precision format under edge-hardware constraints.

The source code of the project developed for this dissertation is publicly available at the link below.

`https://github.com/SebastiaoPFerreira/projeto_TFM_LPR_Edge`

1.4 Document Organization

This thesis is divided into six chapters.

- Chapter 1 – Introduction: Presents the motivation, context, problem statement, objectives, and research outcomes of the project.
- Chapter 2 – State of the Art in License Plate Recognition: Reviews the existing literature relevant to the domain, including the fundamentals of the LPR pipeline, deep learning architectures for detection and recognition, edge computing hardware, and related work in edge-based LPR systems.
- Chapter 3 – Methodology and Benchmarking: Describes the proposed two-stage pipeline architecture, the datasets and evaluation metrics used, and the desktop Graphics Processing Unit (GPU) benchmarking process used to select the detection and recognition models carried forward for edge deployment.
- Chapter 4 – Model Optimization for Khadas VIM3: Details the conversion, compilation, and precision optimization of the selected models for the Khadas VIM3's Neural Processing Unit (NPU), and describes the resulting hardware-aware end-to-end pipeline.
- Chapter 5 – Experimental Evaluation and Results: Presents the measured detection, recognition, and end-to-end system performance of the deployed pipeline on the Khadas VIM3, and compares these results against the related work.
- Chapter 6 – Conclusion and Future Work: Summarizes the contributions of the thesis and discusses directions for future work.

2 STATE OF THE ART IN LICENSE PLATE RECOGNITION

2.1 Background

This chapter provides the theoretical foundation and a review of the state-of-the-art in license plate recognition. It describes the fundamental pipeline of ALPR systems, the architectural evolution of detection and recognition models, and addresses the paradigm shift toward edge computing.

2.1.1 Fundamentals of License Plate Recognition

Automatic License Plate Recognition is an application of computer vision designed to extract information from vehicle license plates. The architecture of these systems is typically conceptualized as a sequential pipeline that transforms unstructured visual data into structured text through a series of processing stages.

As illustrated in Figure 2.1, the standard LPR pipeline consists of six stages: image acquisition, pre-processing, license plate detection, character segmentation, OCR, and post-processing [33].

Implementations of the **recognition stage** generally fall into two categories: Multi-Stage and Single-Stage (segmentation-free) architectures. Traditional Multi-Stage systems explicitly separate character segmentation from optical character recognition, whereas Single-Stage approaches aim to process the cropped license plate as a unified object. Single-Stage or segmentation-free approaches aim to process the license plate as a unified object. These methods often bypass the explicit segmentation of individual characters, mapping the entire license plate image directly to a text sequence, thereby reducing processing time and eliminating errors caused by poor character separation. However, this unified approach introduces specific trade-offs. Unlike multi-stage pipelines, which are highly modular and allow for easier debugging of individual steps (e.g., verifying if segmentation failed before recognition), single-stage models are harder to debug making error analysis more complex. Furthermore, when implemented with deep learning, they typically require significantly larger datasets of annotated full license plates for training, and often

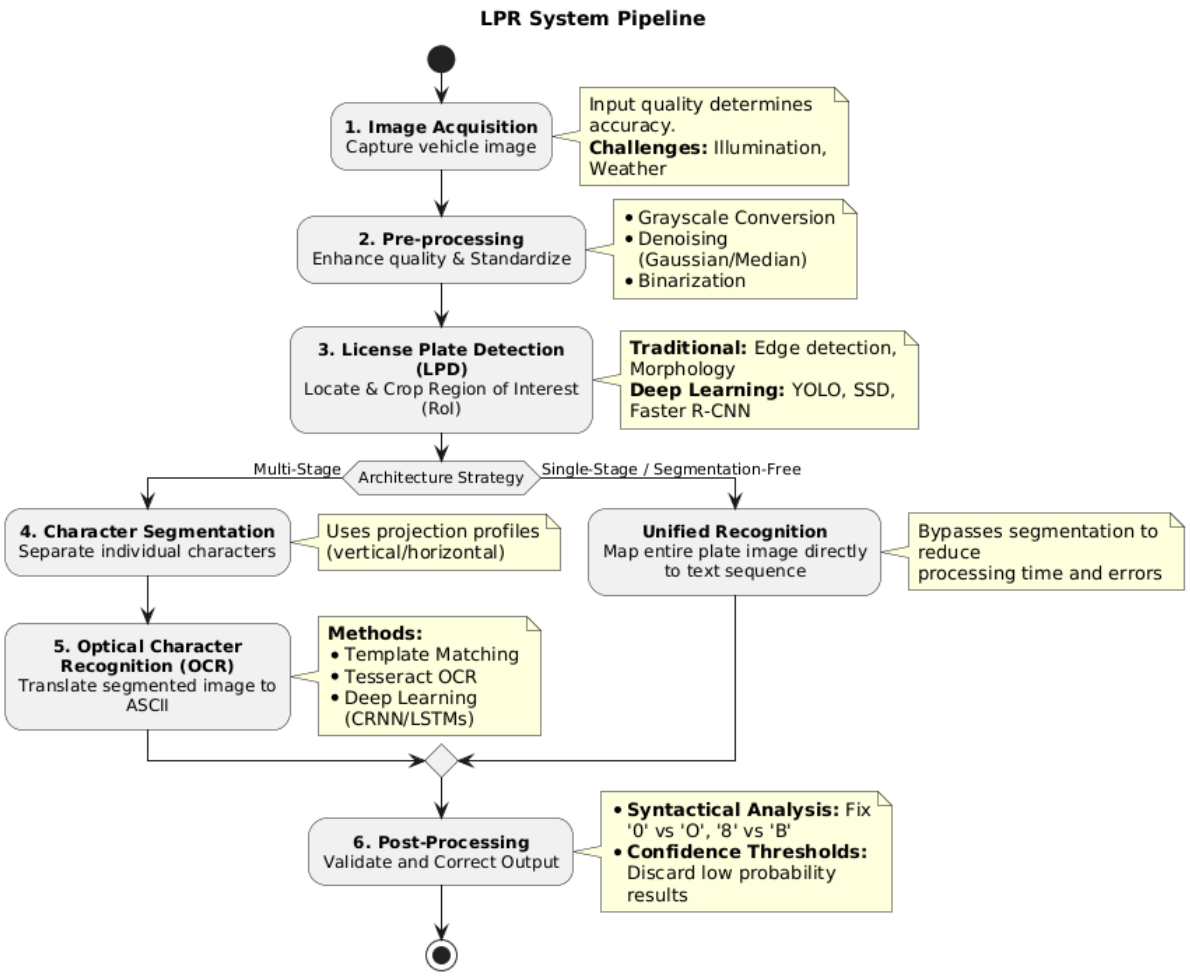


Figure 2.1: *Overview of the LPR System Pipeline*

suffer from lower accuracy when handling high-resolution details or severely distorted plates compared to methods that isolate and rectify each character individually.

The first stage of the pipeline is image acquisition, which involves capturing a picture of a vehicle. The quality of the input data directly impacts the system's overall accuracy. In real-world scenarios, this capture can be influenced by environmental challenges such as varying illumination and weather conditions.

The acquired raw image often contains noise and artifacts that can hinder detection accuracy. Therefore, pre-processing techniques are applied to enhance the image quality and standardize the input for the detection algorithms. These techniques typically consist of:

- **Grayscale Conversion**, which converts RGB images into grayscale to reduce the dimensionality of the data and consequently lower the computational load;

- **Denoising**, that applies filters, such as gaussian or median blurring, to remove high-frequency noise caused by sensor grain or environmental factors;

- **Normalization and Binarization**, which are used to convert grayscale images into binary (black and white) formats. Doing this increases the contrast between the characters and the background of the plate, which helps the detection algorithms.

After pre-processing the image, the next step is to detect the license plate within the larger vehicle image and extract its Region of Interest (RoI).

Traditional image processing methods relied on rule-based algorithms to detect plates using geometric and contrast properties. Common techniques include canny edge detection, morphological operations, heuristics filtering, and Hough transform.

- Canny Edge Detection: This algorithm identifies the sharp, high-contrast boundaries of a license plate by applying a gaussian filter to smooth the noise, to detect strong edges while suppressing weaker ones;

- Morphological Operations: Operations such as dilation and erosion were employed to enhance the plate regions and connect broken character segments, helping to distinguish the plate's rectangular structure from the vehicle background;

- Heuristics and Hough Transform: Systems based on heuristics and Hough transform use line detection to identify the straight edges of a plate's borders, while heuristic constraints, such as aspect ratio and pixel density, were applied to filter out non-plate regions like headlights or radiators.

While computationally inexpensive, these methods struggled with complex backgrounds, varying illumination, and oblique angles.

Recently, deep learning models have become the industry standard for this task. Although computationally more expensive than the classical techniques described above, they have proven to be significantly more reliable, maintaining high accuracy even in unconstrained environments where traditional methods fail.

Once the license plate has been detected and cropped, the next step is character segmentation. This process involves separating the individual characters from the license plate background to prepare them for recognition. Techniques for this segmentation often rely on projection profiles (analyzing the distribution of pixel intensities horizontally and vertically) to find the gaps between characters.

The recognition stage translates the segmented characters into American Standard Code for Information Interchange (ASCII) text. Classical approaches utilized template matching, where the segmented character image was compared pixel-by-pixel against a database of reference fonts to find the highest correlation.

Recently, the standard has shifted towards deep learning methodologies. Modern OCR engines now predominantly utilize deep learning models to learn character

representations directly from data. Deep learning models can generalize across a vast range of fonts and environmental conditions, providing higher accuracy in real-world scenarios.

The final stage involves validating and correcting the output from the OCR engine to improve system reliability. Since license plates follow specific rules based on their region of origin (e.g., specific arrangements of letters and numbers), this domain knowledge can be used to filter out false positives. For instance, syntactical analysis can correct common OCR errors, such as distinguishing between the digit '0' and the letter 'O', or '8' and 'B', based on their position in the string. Confidence thresholds are also applied here; if the probability score is below a certain limit, the system may discard the result to prevent false alerts.

2.1.2 Deep Learning

Deep Learning is a specialized subset of Machine Learning (ML) based on Artificial Neural Networks (ANN). While traditional ML often relies on manual feature extraction, deep learning algorithms automate this process. They are capable of learning data representations with multiple levels of abstraction, obtained by composing simple but non-linear modules that transform the raw input into a representation at a higher abstract level.

To better understand the architecture of modern LPR systems, it is essential to distinguish between the primary types of Deep Learning models: Convolutional Neural Networks, Recurrent Neural Networks, and Transformers.

2.1.2.1 Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNN) are the dominant architecture for processing grid-like data, such as images. A standard CNN consists of three main types of layers:

- **Convolutional Layers:** These apply a set of learnable filters to the input image to create feature maps. These filters automatically learn to detect visual patterns, such as edges, textures, and shapes;
- **Pooling Layers:** These reduce the spatial dimensions of the feature maps (downsampling), decreasing the computational power required and making the network invariant to small translations and distortions in the image;
- **Fully Connected Layers:** Located at the end of the network, these layers map the extracted features to final output classes (e.g., classification scores for bounding boxes).

In the context of LPR, CNN form the backbone of object detectors.

2.1.2.2 Recurrent Neural Networks (RNNs)

While CNN excel at spatial processing, Recurrent Neural Networks (RNNs) are designed for processing sequential data. Unlike standard feedforward networks, RNNs have memory loops in the network architecture that allow using past information. This makes them suitable for tasks where the context of previous inputs influences the current output, such as speech recognition or natural language processing.

Standard RNNs suffer from the "vanishing gradient"problem, making it difficult to learn long-term dependencies. To address this, variants like Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) were developed. In LPR systems, RNNs (specifically Bi-directional LSTM) are frequently used in the OCR stage to recognize the sequence of characters on a plate, where the probability of a character often depends on its neighbors (e.g., distinguishing syntax patterns).

2.1.2.3 Transformers

The Transformer architecture represented a paradigm shift in deep learning. Originally designed for Natural Language Processing, Transformers discard the recurrence of RNNs entirely, relying instead on the self-attention mechanism. This mechanism allows the model to weigh the importance of different parts of the input data relative to each other. This can be done in parallel, rather than sequentially like in the Recurrent Neural Network (RNN).

Recently, this architecture has been adapted for computer vision (Vision Transformer - ViT [4]). By treating an image as a sequence of patches, transformers capture global dependencies across the entire image.

In modern LPR, Transformers are increasingly replacing RNNs in the OCR stage due to their ability to handle irregular text arrangements and their higher parallelization capabilities on GPU hardware.

2.1.3 Deep learning for LPR

Deep learning-based object detection models have become the standard for the license plate detection stage. The models generally fall into two architectural types:

- **Two-Stage Detectors:** Models like Faster R-CNN [25] operate by first generating a sparse set of region proposals and then classifying these regions. Faster R-CNN has a high localization accuracy, making it particularly effective for detecting small or distant plates. However, this two-step process involving a region proposal network followed by a classification head is computationally intensive, often resulting in low frame rates;
- **Single-Stage Detectors:** To address the latency issues of two-stage models, single-stage architectures like YOLO [23] and Single Shot MultiBox Detector

(SSD) [15] were developed. These models treat object detection as a single regression problem, predicting bounding box coordinates and class probabilities directly from image pixels in one forward pass. The YOLO family, in particular, offers a high speed-accuracy ratio.

2.1.4 OCR using CNN and Transformer-based Models

Once the license plate has been localized, the challenge shifts to recognizing the sequence of characters. Deep learning has introduced powerful sequence recognition architectures that outperform traditional template matching.

- **Recurrent Neural Networks (LSTM/CRNN):** These models combine a CNN backbone (to extract visual features) with Bidirectional Long Short-Term Memory (BiLSTM) networks. The RNN models the contextual dependencies between characters. The network is trained using the Connectionist Temporal Classification (CTC) [8]. CTC addresses the alignment problem in sequence by calculating the probability of all possible label alignments given the input sequence. This allows it to predict character sequences directly from the input image without requiring pre-segmented character labels or explicit timing information;
- **Vision Transformer (ViT):** This architecture applies the self-attention mechanisms directly to the OCR problem. By capturing global dependencies across the entire image simultaneously, rather than processing data sequentially, ViT focus on relevant parts of the character image regardless of their position, resulting in improved performance against distorted, rotated, or irregular text found in real-world scenarios.

2.1.5 Importance of Datasets in Deep Learning

The effectiveness of the deep learning architectures discussed previously, ranging from detection models to recognition engines requires vast amounts of labeled data to learn hierarchical representations resilient to environmental variabilities. Consequently, the availability of large-scale, annotated datasets has become an important requirement for modern LPR development.

To achieve high accuracy in "real-world"scenarios, datasets must encompass a wide range of conditions. This includes variations in illumination (day vs. night), weather (rain, fog), and camera angles. Furthermore, datasets for LPR are typically categorized based on the specific stage of the pipeline they address. Detection Datasets contain annotations for the bounding boxes of license plates within full vehicle images,and Recognition (OCR) Datasets focus on the character sequences themselves, often requiring cropped images of plates with corresponding text labels.

2.1.6 Edge Computing

The increase of compute-intensive applications to support smart cities, ranging from real-time traffic control to automated security monitoring, has determined the necessary migration of computing from the cloud to the edge.

2.1.6.1 Benefits for LPR

Deploying LPR systems at the edge offers several advantages over cloud-based implementations, mainly those regarding performance and efficiency. A major benefit is the reduction in latency. Real-time applications, such as identifying fleeing vehicles, require immediate responses. Processing data locally eliminates the transmission delays inherent in cloud architectures, reducing overall system latency. This local processing also minimizes network bandwidth usage, as the system transmits only the extracted metadata (e.g. the license plate string) rather than continuous high-definition video streams.

Beyond performance, edge computing enhances security and scalability. Processing video on-device ensures that sensitive surveillance information does not need to traverse the public internet, thereby mitigating privacy concerns and reducing the risk of malicious attacks. Furthermore, the energy efficiency and lower cost of edge devices facilitate mass scalability in smart cities. These systems can be deployed in remote or rural locations, such as forests or isolated highways, where they can operate independently on battery power without relying on a direct power grid or constant cloud connectivity.

2.1.6.2 Edge AI Hardware

To execute deep learning models in resource-constrained environments, there are several specialized embedded devices that can be considered, with different tradeoffs between computational performance and power consumption. At the high-performance end, there is the NVIDIA Jetson family, which provides embedded boards equipped with powerful embedded GPU designed specifically to accelerate deep learning tasks.

For applications where energy efficiency is paramount, Application-specific integrated circuits (ASIC) such as the Google Coral Tensor Processing Unit (TPU) [22] offer an alternative to GPU. Built around the Edge TPU architecture [26], this hardware utilizes a systolic array design that executes massive parallel matrix multiplications. This allows the device to perform inference at high speeds while consuming minimal power.

Offering a unique advantage in terms of adaptability, Field Programmable Gate Array (FPGA) based accelerators utilizing stacks like Xilinx Vitis AI [14], allow for hardware-level reconfiguration. Unlike fixed-function processors (CPU/GPU), FPGA enable developers to design custom data paths tailored to specific neural

network layers.

Finally, ARM-based smart cameras and Internet of Things (IoT) devices provide a cost-effective solution for widespread deployment. Devices such as the Raspberry Pi or specialized smart cameras utilize standard ARM Cortex processors to perform on-sensor computing, processing video frames locally before transmission. While these single-board computers possess limited raw processing and power compared to server-grade hardware, they are sufficient for executing highly optimized, lightweight LPR models in decentralized edge hierarchies. Specifically, they serve effectively in 'Low-tier' networks acting as independent sensor endpoints, and 'Mid-tier' networks functioning as local gateways that aggregate data from multiple simpler sensors.

2.1.7 Applications of LPR

The utility of ALPR systems extends far beyond vehicle identification. In the context of the ITS, LPR technology is applied to diverse sectors ranging from traffic engineering to environmental protection. By converting vehicle registration data into digital intelligence, these systems enable automation in scenarios that previously required manual intervention. The following subsections detail the primary domains where LPR is currently deployed, highlighting its role in enhancing urban mobility, public safety, and infrastructure efficiency.

2.1.7.1 Smart Surveillance Systems

The integration of LPR into smart surveillance frameworks improved public safety and urban management. Unlike traditional surveillance, which relies on human operators, smart surveillance systems utilize computer vision to actively interpret video data in real-time.

In the domain of traffic engineering, LPR enables a detailed analysis of traffic flow dynamics, allowing systems to calculate average travel times between points and identify congestion bottlenecks. This capability can be integrated with automated enforcement mechanisms, where the system autonomously detects violations, such as speeding, red-light infractions, or the unauthorized use of bus lanes without human intervention. Simultaneously, this real-time identification serves as a critical tool for public safety by cross-referencing license plates with national databases law enforcement agencies can receive instant alerts for vehicles of interest, such as stolen cars or flagged suspects, enabling rapid response to criminal activity.

2.1.7.2 Electronic Toll Collection and Environmental Zones

Another application of LPR technology is in Electronic Toll Collection (ETC) and congestion pricing. Unlike traditional toll plazas, LPR-based systems eliminate the need for physical barriers or Radio Frequency Identification (RFID) tags. In smart

parking facilities, these systems facilitate entry by automatically raising barriers upon recognizing authorized plates or calculating precise parking fees based on entry and exit timestamps.

2.2 Related Work in License Plate Recognition

This section reviews the evolution of ALPR methodologies. The research is broadly categorized into traditional computer vision approaches, which rely on handcrafted features, and modern deep learning-based systems that utilize learnable feature representations [19].

2.2.1 Traditional LPR Systems

Early implementations of LPR systems were predominantly based on conventional image processing techniques and rule-based algorithms [1]. These methods typically approach the problem by exploiting the specific geometric and textural properties of license plates, such as their high contrast, rectangular shape, and specific aspect ratios.

Classical Recognition (OCR)

For character recognition, these systems primarily utilized Template Matching [9]. In this approach, segmented characters were normalized to a fixed size and compared pixel-by-pixel against a pre-defined database of font templates. Classification was determined by calculating the correlation coefficient or minimizing the Hamming distance between the input and the reference templates.

While computationally inexpensive, traditional LPR systems exhibited significant limitations in "real-world"scenarios:

- **Poor Robustness to Environmental Conditions:** Rule-based algorithms struggled severely with varying illumination. Shadows, glare, and low-light conditions often fragmented the edge features necessary for detection [19] [1].
- **Geometric Rigidity:** Techniques like Template Matching, which identify characters by comparing input pixels against a set of fixed reference images, require precise character segmentation and alignment. They were unable to handle geometric distortions, such as oblique camera angles (skew) or plate rotation, leading to high recognition failure rates [33] [19].
- **Noise Sensitivity:** Morphological operations were prone to generating false positives in complex backgrounds (e.g., radiator grilles or bumper stickers) that mimicked the texture of a license plate [19].

These limitations necessitated the shift towards data-driven approaches, which are discussed in the following subsections.

2.2.2 Relevant LPR Architectures

The YOLO family is dominating the detection stage by treating object localization as a single-stage regression problem. Architecturally, these models consist of three primary processing stages, a feature extractor to capture visual patterns, a feature aggregation module to fuse information across different images scales and a final detection layer that outputs bounding box coordinates and class probabilities. While early iterations such as YOLOv3 [24] provided a reliable baseline, the field has rapidly moved around the latest variants, specifically YOLOv8 [12] and the emergent YOLOv10 [34] and YOLOv11 [13] architectures, as the optimal choices for modern surveillance.

YOLOv8 represents a paradigm shift for edge deployment by introducing an anchor-free detection architecture. Unlike previous versions that relied on matching objects to pre-defined reference rectangles, so called anchor boxes, YOLOv8 treats detection as a direct point-prediction task. Identifies the center of the object on the feature map and directly regresses the distances to the four boundaries of the bounding box. This architectural design effectively optimizes the trade-off between computational efficiency and detection precision, making it the current standard-bearer for practical applications [17].

Building on this foundation, YOLOv10 and YOLOv11 integrate advanced features such as Non-Maximum Suppression (NMS)-free (Non-Maximum Suppression free) training. In conventional detectors, the model predicts thousands of overlapping boxes for a single object, requiring a computationally expensive post-processing step (NMS) to filter duplicates. NMS-free training teaches the network to predict exactly one bounding box per object, removing this sorting process entirely and reducing inference latency. These architectural reforms significantly reduce the computational overhead during post-processing, enabling higher frame rates on resource-constrained embedded devices while maintaining superior mean Average Precision (mAP) even in cluttered or fast-moving traffic scenes[34].

Once the license plate is localized, the recognition stage is typically handled by one of these approaches, each suiting different constraint profiles. The Convolutional Recurrent Neural Network (CRNN) [28] is a high-accuracy text recognition system, by combining a CNN backbone (like ResNet [10]) with BiLSTM, it models the semantic dependencies between characters. Alternatively, Tesseract OCR [29] serves as a baseline open-source engine, utilizing a hybrid approach that combines traditional computer vision with LSTM networks. In this architecture, the LSTM processes the text line as a sequence of feature slices rather than isolated characters,

allowing the engine to use context to resolve ambiguities (e.g., distinguishing '5' from 'S'). It offers a solution that does not require the extensive training data needed for custom models, although it is computationally heavier compared to purpose-built lightweight networks.

Another widely adopted general-purpose alternative is EasyOCR [11], an open-source, deep learning-based OCR engine supporting a broad range of languages and scripts out of the box. Like Tesseract, it is not purpose-built for license plates and instead trades specialization for versatility, making it a useful general-purpose baseline for comparison against domain-specific recognition architectures.

For environments where latency is the primary constraint, LPRNet [38] offers a streamlined alternative to both CRNN and Tesseract. With an optimized CNN in the backbone and a CTC head, LPRNet achieves fast inference speeds suitable for low-power hardware, without compromising accuracy on standard plate layouts.

Finally, the industry is increasingly moving towards unified End-to-End frameworks that dissolve the traditional boundaries between detection and recognition. Models like ALPRNet [16] integrate both tasks into a single forward pass, preventing the error propagation that often occurs in multi-stage pipelines. Similarly, PaddleOCR [3] has emerged as a versatile, ultra-lightweight solution. It employs a two-stage logic handling multi-language text and perspective distortions. This makes it particularly effective for real-world scenarios where vehicles are captured at oblique angles or under challenging lighting conditions.

Table 2.1: Summary of Baseline Performance Metrics Reported in Literature

Study	**Architecture**	**Reported Accuracy**	**Reported Latency**	**Hardware**
Jocher et al. [12]	YOLOv8 (nano)	37.3% mAP@50-95 (COCO val2017)	0.99 ms	NVIDIA A100 (TensorRT)
Zherzdev & Gruzdev [38]	LPRNet	Up to 95% (Chinese license plates)	1.3 ms / plate	Intel Core i7-6700K (CPU)
Tykva [33]	Faster R-CNN + Custom HTR	98.15% AP (detection) / 0.58% CER (recognition)	∼71 ms (detection) / 23.65 ms (recognition)	Tesla P100 (detection) / GeForce 930MX (recognition)

Table 2.1 lists the baseline performance reported in the literature for the architectures relevant to this work. While the reported inference latencies for models like YOLOv8 (0.99 ms) and LPRNet (1.3 ms) are exceptionally low, these figures are achieved on server-grade data center hardware, such as an NVIDIA A100 GPU, or desktop-class processors like the Intel Core i7. Similarly, while Tykva [33] achieves excellent accuracy across a multi-stage pipeline, the system relies on a Tesla P100 and a discrete GeForce 930MX GPU. These hardware environments do not reflect the strict computational, memory, and power constraints of embedded edge devices.

Consequently, while these literature baselines serve as an architectural reference, they demonstrate the clear need for the independent, edge-focused benchmarking methodology detailed in Chapter 3, which isolates how these architectures actually perform when migrated to a constrained, mobile-grade System-on-Chip.

2.2.3 Edge-Based LPR Systems

2.2.3.1 GPU-Accelerated Edge Devices (NVIDIA Jetson)

The NVIDIA Jetson [18] family (Nano, TX2, Xavier, Orin) has emerged as a good platform for evaluating edge-LPR performance due to its embedded GPU capabilities. Numerous studies [30] [17] have demonstrated the efficacy of deploying "Nano"or "Small"variants of YOLO on these devices. Research indicates that models like YOLOv8n (Nano), when deployed on a Jetson Nano using TensorRT optimization a proprietary Software Development Kit (SDK) that accelerates deep learning inference by fusing network layers and optimizing memory usage, can achieve inference speeds exceeding 30 FPS [36]. This performance is important for high-speed traffic monitoring, a domain where heavier two-stage detectors like Faster R-CNN often fail to sustain the frame rates required for dynamic scenes. The specific throughput required to qualify as "real-time,"however, is application-dependent, varying with vehicle speed, camera field of view, and deployment context, rather than a single fixed threshold.

To complement fast detection, researchers frequently pair these detectors with highly optimized recognition networks [35]. Implementations utilizing LPRNet on Jetson hardware have shown the ability to perform end-to-end recognition with minimal latency, validating the feasibility of fully autonomous, battery-powered enforcement units [35].

2.2.3.2 FPGA-Based Accelerators

While GPU provide accessibility, recent literature investigates FPGA for scenarios requiring deterministic latency and extreme energy efficiency.

The state-of-the-art in FPGA-based LPR focuses on overcoming hardware constraints through aggressive model compression. Recent studies [2] have demonstrated the successful deployment of quantized YOLO variants directly onto FPGA logic fabrics.

Comparative studies [7] between FPGA and embedded GPU implementations report superior energy-efficiency ratios for FPGA. Specific FPGA designs have been shown to process video streams at frame rates comparable to the NVIDIA Jetson Nano but with significantly lower power consumption [31]. This characteristic is cited as a decisive factor for deploying autonomous, solar-powered installation sites where the energy budget is strictly limited.

2.2.4 Performance Benchmarks

When evaluating object detection models for edge deployment, it is essential to understand metrics used to balance accuracy against speed and efficiency. The most common metris are:

- **Mean Average Precision (mAP):** This is a metric for measuring the prediction accuracy in object detection. It is derived from two values:
  - **Precision (P):** The percentage of positive detections that are correct.

$$P = \frac{TP}{TP + FP} \tag{2.1}$$

  - **Recall (R):** The percentage of actual objects that were correctly detected.

$$R = \frac{TP}{TP + FN} \tag{2.2}$$

  where TP is True Positives, FP is False Positives, and FN is False Negatives.

  The **Average Precision (AP)** is calculated as the area under the Precision-Recall curve ($P(R)$) for a single class:

$$AP = \int_0^1 P(R)\, dR \tag{2.3}$$

  The **mAP** is simply the mean of the AP values across all N classes (or IoU thresholds):

$$mAP = \frac{1}{N} \sum_{i=1}^{N} AP_i \tag{2.4}$$

  In LPR, a high mAP is crucial to minimize both false positives, detecting text where there is none, and false negatives, missing a license plate entirely.

- **Confidence Thershold (τ):** Deep learning models output a probability score representing the certainty that a detected object belongs to a specific class. The threshold is a manually defined cutoff value that determines which predictions are accepted

- **Latency and Frames Per Second (FPS):** These two metrics are inversely related and define the system's temporal performance. Latency measures the time required to process a single image (inference time), typically in milliseconds. FPS indicates the throughput, or how many images the system can handle per second. For high-speed scenarios, such as highway monitoring, low latency is required to maintain a high FPS, ensuring the system does not miss vehicles moving quickly through the frame.

2.2.5 Commercial and Open-Source Industry Solutions

Beyond the academic architectures discussed previously, several robust solutions have established themselves as industry standards for rapid LPR deployment. These engines often prioritize versatility and multi-language support, aiming to provide immediate utility for developers without the need for extensive model training.

Among general-purpose recognition engines, there is Tesseract [29] that stands as a baseline open-source recognition system, that offers broad language support, capable of recognizing over 100 languages, making is highly effective for document scanning, and running on standard CPU. However, its generalized nature often proves to be a bottleneck for real-time video, it is computationally heavier and generally slower on embedded devices compared to purpose-built lightweight networks. Consequently, while it serves as a reliable baseline, it is frequently outperformed by specialized deep learning models in high-speed traffic scenarios.

To bridge the gap between complex academic models and strict industrial edge constraints, hardware manufacturers, such as NVIDIA and Texas Instruments, have released collections of pre-trained models specifically optimized for their accelerators. These repositories offer validated versions of lightweight detectors, such as YOLO-Lite [21] or LPRNet [38], which are specifically tuned for their respective hardware accelerators.

2.3 Discussion and Research Gap

The review of the state-of-the-art reveals a decisive shift in ALPR. The industry has effectively transitioned from rule-based image processing to data-driven deep learning methodologies that offer an advantage against noise, lighting changes, and geometric distortions.

Regarding detection architectures, the literature highlights a clear shared conclusion. While two-stage models, like Faster R-CNN, set the benchmark for precision, their computational overhead often limits performance, rendering them unsuitable for the real-time demands of modern edge surveillance. In contrast, the research community and industry have focused on single-stage detectors. Modern variants, particularly YOLOv8 and YOLOv11, have effectively closed the accuracy gap while delivering the high throughput necessary for capturing fast-moving vehicles. This shift allows for the deployment of sophisticated detection algorithms on resource-constrained embedded devices without compromising the system's ability to respond to traffic events in real-time.

In the recognition stage, the trade-off predominantly lies between versatility and inference speed. Heavy engines provide broad utility but create latency bottlenecks

on embedded hardware. Consequently, segmentation-free architectures, such as LPR-Net and CRNN-CTC, have become the preferred choice for edge-native applications. By eliminating the costly pre-segmentation steps of traditional pipelines and regressing the entire license plate string directly from global image features, these models reduce computational overhead.

Despite these advancements, a gap remains in optimizing these deep learning pipelines for edge constraints. While nano models exist, integrating detection, tracking, and recognition on battery-powered devices without significant accuracy loss remains a challenge. Most existing commercial solutions either rely on cloud connectivity, which introduces latency and privacy risks, or require powerful edge GPU that exceed the power budget of remote, off-grid installation sites.

3 METHODOLOGY AND BENCHMARKING

This chapter details the methodological framework proposed for the development of the edge-based LPR system. It outlines the specific challenges addressed, provides a high-level overview of the system architecture, and presents preliminary experimental results that justify the selection of the core detection and recognition algorithms.

The goal of the methodology is to isolate two independent variables that are frequently conflated in the literature, architectural accuracy of a given detection or recognition model, and the practical performance obtained once that model is subjected to the constraints of embedded hardware. To this end, the benchmarking strategy is structured as a progressive narrowing process. Candidate architectures are first evaluated on a desktop GPU to establish an accuracy and latency upper bound, free from any hardware-imposed compromise. The surviving candidates are then exported to the Open Neural Network Exchange (ONNX) format and executed on the target device's ARM CPU using ONNX Runtime, which yields an unaccelerated but representative edge baseline. This two-tier evaluation isolates the cost, in both latency and accuracy, of moving from a server-grade environment to constrained hardware, and produces the empirical justification required to select the models that are subsequently optimized for NPU execution in Chapter 4.

The target platform for this work is the Khadas VIM3, a single-board computer built around the Amlogic A311D System-on-Chip (SoC). This SoC combines a general-purpose ARM CPU cluster with a dedicated NPU rated at 5.0 Tera Operations Per Second (TOPS), positioning it as a representative example of the low-cost, ARM-based edge devices discussed in Section 2.1.6.2. All benchmarking activities described in this chapter were designed with this heterogeneous architecture in mind, since the final deployment pipeline distributes computation across both the CPU and the NPU, accessed respectively through ONNX Runtime and the Khadas Acuity Toolkit / Khadas Sensor Neural Network (KSNN) Application Programming Interface (API), rather than relying exclusively on either.

3.1 Proposed Architecture for LPR

As introduced above, the Khadas VIM3 provides two independent compute domains: a general-purpose ARM CPU cluster and a dedicated NPU. The proposed system follows a two-stage pipeline architecture. This design choice was deliberate. Although Single-Stage or unified end-to-end architectures reduce error propagation in principle, this two-stage approach preserves the modularity required to profile, debug, and independently optimize the detection and recognition stages across these two compute domains. It also allows each stage to be benchmarked and swapped independently during the model selection process, without requiring a full pipeline retrain whenever a candidate architecture is changed. Figure 3.1 illustrates this proposed architecture, showing the Detection and Recognition stages as independent modules connected only by the cropped RoI.

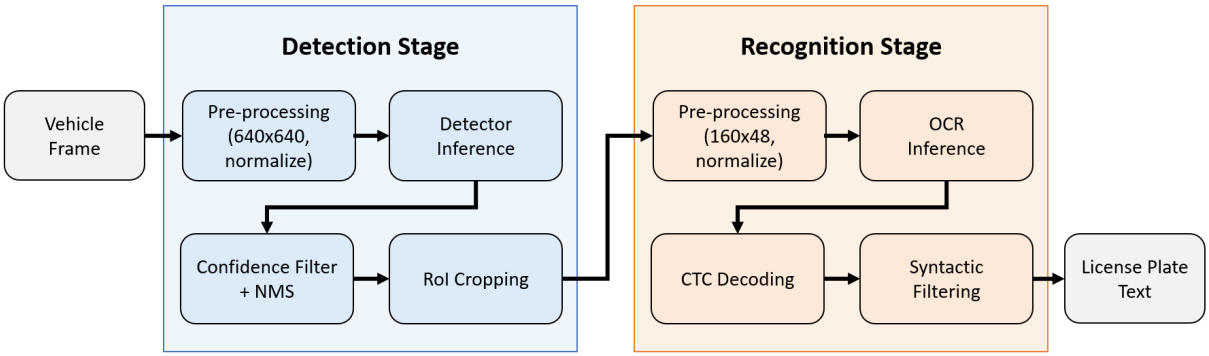


Figure 3.1: *Proposed Two-Stage Architecture for the Edge-Based LPR Pipeline*

The **detection stage** is responsible for locating the license plate region within the full vehicle frame. Input images are resized to a fixed resolution of 640×640 pixels and normalized to the [0.0, 1.0] range prior to inference, matching the input expected by the YOLO detector. The candidate architectures for this stage are drawn from the YOLOv8-and-later generation of the YOLO nano family, whose anchor-free, single-pass regression design and reduced parameter count, discussed in Section 2.2.2, make these variants the most promising for NPU deployment. Several YOLO nano variants, together with heavier alternatives (Faster R-CNN and SSD), are benchmarked in Section 3.4 to confirm this assumption empirically rather than by architectural reasoning alone. The specific variant ultimately deployed is determined only after this benchmarking, together with the further precision and compatibility driven testing described in Chapter 4.

On the target device, the detection stage itself is split across CPU and the NPU. The YOLO forward pass executes on the NPU, producing three-scale feature maps corresponding to the multi-resolution detection heads typical of the YOLO architecture. The decoding of these raw feature maps into candidate bounding boxes is performed on the CPU. This post-processing stage applies two filtering

steps. The first is a confidence threshold, which discards low-probability detections and must be selected to balance the risk of missed detections, when set too high, against false positives, when set too low. The second is NMS, governed by an Intersection over Union (IoU) threshold, which eliminates duplicate, overlapping boxes corresponding to the same physical plate. Both parameters directly influence downstream recognition performance, an overly permissive confidence threshold increases the number of invalid RoIs passed to the recognition stage, while an overly strict IoU threshold risks discarding a valid detection in favour of a near-duplicate box. Once a bounding box survives post-processing, the corresponding RoI is cropped directly from the original, full-resolution vehicle image, ensuring the license plate crop retains the maximum available pixel detail before being passed to the recognition stage.

The **recognition stage** translates the cropped RoI into a text sequence. Four candidate recognition engines were established for this stage: Tesseract OCR, EasyOCR, LPRNet, and PaddleOCR. The design decisions described in the remainder of this section correspond to PaddleOCR, the candidate around which the recognition pipeline was built, on account of its lightweight convolutional backbone and its use of CTC decoding, which removes the need for explicit character segmentation.

A further design decision at this stage was to halve PaddleOCR's default recognition input size, from 320×48 to a static 160×48 tensor. Since the computational cost of a convolutional backbone is expected to scale with input width, this reduction was expected to meaningfully lower the recognition stage's arithmetic workload. The actual latency impact of this change is not isolated in this methodology chapter and is instead reflected in the recognition stage results reported in Chapter 5, which measure the pipeline with this input size already applied. Preliminary inspection of the target domain, European license plates, suggested that the fixed aspect ratio and standardized character layout of these plates could tolerate this more aggressive downscaling without a substantial loss of legible character detail, making the trade-off a reasonable candidate for a latency-constrained deployment.

Compiling this architecture for execution on the Khadas VIM3 additionally required a set of graph-level modifications to satisfy the operator constraints of the Khadas Acuity Toolkit. These compilation-specific adaptations, including the ONNX operator set (Opset) requirements and the resulting modifications to the model graph, are compilation methodology rather than pipeline design, and are therefore detailed separately in Section 4.2, together with the equivalent adaptations required for the detection model.

Following model execution, the raw output of the CTC-based recognition head is decoded using a Numpy-based greedy decoding routine. This decoder collapses repeated consecutive predictions and removes the CTC "blank"token, which is explicitly mapped to index 0 of the character dictionary to match PaddleOCR's original

training convention. The resulting character sequence is then passed through a final filtering step using the regular expression `[^A-Z0-9]`, which discards any character outside the alphanumeric range expected on a license plate, providing a lightweight form of the syntactical post-processing described in Section 2.1.1.

3.2 Dataset Selection

Two distinct datasets were required to support the independent evaluation of the detection and recognition stages, curated from publicly available license plate repositories [32, 37].

For the detection stage, a dataset of urban traffic imagery was assembled from these sources, comprising full-frame vehicle images annotated with license plate bounding boxes. Attention was paid to ensuring variability across the conditions for real-world situations, namely differing illumination levels (daytime and low-light conditions), camera angles (including oblique, non-frontal perspectives), and subject distance (near-field and far-field captures). This variability is intended to approximate the operating conditions of a fixed surveillance camera, where vehicles are rarely captured under ideal, front-facing, well-lit conditions.

For the recognition stage, the dataset consists of cropped license plate images extracted from these collections at their native resolution, which varies across samples (for example, dimensions in the region of 300×40 or 540×130 pixels). Rather than requiring a uniformly pre-sized dataset, each crop is dynamically resized to the fixed 160x48 tensor at load time, using area-based interpolation. This interpolation method was selected because it performs a local pixel averaging operation during downscaling, which reduces aliasing artifacts on fine character strokes compared to simpler resampling methods, an important consideration given the aggressive width reduction applied to the recognition input. Ground-truth labels for the recognition dataset are embedded directly in each image's filename (for instance, "12AB34.jpg"), which allows the label to be parsed at load time without requiring a separate annotation file, simplifying the data pipeline used for both training-time evaluation and benchmarking.

3.3 Evaluation Metrics

The general definitions of mean Average Precision (mAP), Precision, Recall, and the latency/throughput relationship were already established in Section 2.2.4 as part of the state-of-the-art review. This section does not redefine these metrics, instead, it specifies which of these standard detection metrics are adopted for this project's evaluation protocol, and introduces the additional metrics required to characterize

the recognition stage and end-to-end hardware performance, neither of which was covered in Chapter 2.

For the detection metrics, following standard object detection practice, the primary metrics adopted from Section 2.2.4 to evaluate the detection stage are Precision, Recall, and mean Average Precision at an IoU threshold of 0.50 (mAP@50). Intersection over Union is additionally reported as a standalone localization metric, since it directly reflects how tightly the predicted bounding box matches the ground-truth plate region, which in turn affects the quality of the RoI passed to the recognition stage.

For the recognition metrics, performance is evaluated using two complementary metrics. Exact sequence match accuracy measures the proportion of predicted plate strings that match the real label character-for-character, reflecting the metric most relevant to a real-world use case, where a partially correct plate reading has limited practical value. To provide a better view of recognition quality, Character Error Rate (CER), calculated between the predicted and real strings, is also used. This second metric captures cases where the recognizer produces a plate string that is close to, but not identical to, the ground truth, which is useful for diagnosing systematic error patterns, for example, consistent confusion between visually similar characters, such as those discussed in Section 2.1.1.

In addition, Character Similarity is reported in several results tables, calculated as the complement of the Character Error Rate ($1 - \text{CER}$), expressed as a percentage. It conveys the same underlying character-level agreement as CER, framed positively as a similarity score rather than an error rate, which is more directly comparable to the percentage-based detection metrics reported elsewhere in this chapter.

Since the central objective of this thesis is real-time performance on constrained hardware, end-to-end inference latency, measured in milliseconds, is decomposed by pipeline stage: detection pre-processing, detection inference, detection post-processing (thresholding and NMS), RoI extraction, recognition pre-processing, recognition inference, and CTC decoding. This decomposition allows latency bottlenecks to be attributed to a specific stage rather than treated as an opaque end-to-end figure, which is useful for identifying where further optimization effort should be directed in Chapter 4. Throughput, expressed in FPS, is derived from the aggregate end-to-end latency and reported as the practical, user-facing performance figure of the system.

3.4 Baseline Model Selection

All preliminary benchmarking described in this section was performed on a dedicated desktop workstation, equipped with an NVIDIA RTX 3060 GPU, kept entirely

separate from the Khadas VIM3 target device.

The selection of the primary edge candidates for each pipeline stage followed a benchmarking-driven process rather than a purely literature-driven one. As a preliminary step, before any NPU-specific conversion work was undertaken, each candidate architecture was evaluated in its native, unquantized form on this desktop platform. This desktop evaluation establishes the architectural upper bound for both latency and accuracy under unconstrained compute, and provides the quantitative justification for narrowing the wider field of candidates discussed in Chapter 2 down to the single detector and recognizer selected for edge optimization in Chapter 4.

3.4.1 Detection Model Benchmarking

Table 3.1 summarizes the average inference latency, measured on the RTX 3060, for the detection architectures considered in this thesis.

Table 3.1: Inference Speed Comparison of Detection Models

Model	Avg Time (ms)	Performance Analysis
YOLOv8n	18.61	High efficiency
YOLOv11n	16.91	Fastest among YOLO models
YOLOv26n	18.60	Competitive performance
SSD MobileNet v3	64.00	Moderate latency, lower efficiency
Faster R-CNN	4573.80	Not suitable for real-time deployment

The results in Table 3.1 provide a confirmation of the architectural trade-off discussed qualitatively in Section 2.1.3. Faster R-CNN, as a two-stage detector, exhibits a latency more than two orders of magnitude higher than any of the single-stage alternatives, a consequence of its sequential region proposal and classification pipeline. While this architecture remains a reasonable reference point for maximum achievable localization accuracy, a latency in excess of 4.5 seconds per frame is incompatible with the real-time surveillance objective. Faster R-CNN was therefore excluded from further consideration as an edge candidate on latency grounds alone.

SSD MobileNet v3 substantially closes this gap, reducing inference time to 64.00 ms, but is nonetheless clearly outperformed in speed by the entire YOLO nano family evaluated, each of which completes inference in under 19 ms.

Table 3.1 reports latency only, since at this stage all five architectures, including the YOLO nano variants, were evaluated using their default, out-of-the-box pre-trained weights, Common Objects in Context (COCO)-pretrained in every case, as noted in Section 3.4, without any fine-tuning on the license plate detection task. Since "license plate"is not among the object classes the COCO dataset defines, none of the five models had a learned concept of a license plate as a detectable object at this stage, and no Precision or Recall figure is reported in Table 3.1 for any of the five architectures.

Because the YOLO nano family already demonstrated a clear latency advantage over SSD MobileNet v3 and Faster R-CNN in Table 3.1, only the YOLO variants were selected for further investigation, each was fine-tuned on a license plate dataset and re-evaluated on accuracy as well as latency, reported in Table 3.2 below. SSD MobileNet v3 and Faster R-CNN, having already been excluded on latency grounds, were not fine-tuned or re-evaluated further, and therefore have no equivalent accuracy figures.

Table 3.2: Detailed Precision and Latency Benchmarks for YOLO Architectures

Model	**mAP@50 (%)**	**Precision (%)**	**Recall (%)**	**Latency (ms)**
YOLOv8n	90.32	96.46	86.29	10.77
YOLOv11n	89.02	97.30	83.29	10.42
YOLOv26n	91.10	96.10	88.24	9.67
YOLOv26s	87.60	96.02	85.29	15.38

Table 3.2 confirms that all three nano-scale YOLO variants achieve broadly comparable detection accuracy on the fine-tuned task, with mAP@50 ranging from 89.02% (YOLOv11n) to 91.10% (YOLOv26n), a spread of only 2.08 percentage points. YOLOv26n records the highest mAP@50, the highest recall, and the lowest latency of the three nano variants in this table. YOLOv11n achieves the highest precision of the four models (97.30%) but the lowest recall of the three nano variants, indicating a more conservative detector that produces fewer false positives at the cost of missing a comparatively larger share of true plates. YOLOv26s, the small variant of the newest architecture, is included as a reference point for the accuracy-latency trade-off within a single YOLO generation: despite its larger size, it records the lowest mAP@50 of the four models (87.60%) at nearly double the latency of YOLOv26n (15.38 ms versus 9.67 ms), confirming that the additional capacity of the small variant is not justified for this task relative to its nano counterpart, and it was not considered further as a deployment candidate.

Consequently, two primary candidates were retained: YOLOv26n and YOLOv8n. YOLOv26n was selected as the strongest overall performer across mAP@50, recall, and latency. YOLOv8n was retained as a highly established, reliable fallback, trailing YOLOv26n's mAP@50 by only 0.78 points. YOLOv11n was eliminated, its mAP@50 and recall fell below YOLOv26n's, and its initial latency advantage over YOLOv26n in the un-tuned benchmark (16.91 ms versus 18.60 ms) reversed post-fine-tuning (10.42 ms versus 9.67 ms). The final selection between YOLOv26n and YOLOv8n is addressed in Chapter 4, through further pruning and precision-based testing, together with the practical compatibility constraints imposed by the Khadas Acuity Toolkit.

3.4.2 Recognition Model Benchmarking

Table 3.3 summarizes the out-of-the-box performance of the four recognition engines considered, using each model's publicly available, pre-trained weights without any project-specific fine-tuning.

Table 3.3: Baseline Performance of Out-of-the-Box Recognition Engines

Engine	Exact Match (%)	Char Similarity (%)	Avg Time (ms)
TesseractOCR	5.33	29.50	129.99
EasyOCR	21.33	66.30	51.97
LPRNet	1.00	47.50	6.57
PaddleOCR	44.67	79.69	37.92

The recognition benchmark reveals a markedly different trade-off pattern than the detection stage, since none of the four engines had been fine-tuned on the target domain of European license plates prior to this evaluation. Tesseract OCR, consistent with its characterization in Section 2.2.5 as a generalized, document-oriented engine, proved both the slowest of the four models tested (129.99 ms) and among the least accurate (5.33% exact match), confirming that its broad, multi-language design does not translate into a competitive baseline for short, structured alphanumeric sequences under real-time constraints.

The most striking result in Table 3.3 is the near-total failure of LPRNet, which, despite recording by far the lowest inference latency of the four engines (6.57 ms), achieved an exact match accuracy of only 1.00%. This outcome is attributed to a domain shift between the model's pre-training distribution and the target use case. The publicly available LPRNet weights used in this benchmark were pre-trained predominantly on the Chinese City Parking Dataset (CCPD) license plate corpus, whose plates follow a character layout, font set, and, critically, an alphabet (including Chinese provincial characters) that differs substantially from the Latin alphanumeric layout used on European plates. Consequently, LPRNet's raw inference speed, while highly attractive for an edge deployment, could not be leveraged out of the box, and this result underscores that recognition latency figures alone are an insufficient basis for model selection when a significant domain gap is present.

EasyOCR achieved a moderate exact match accuracy of 21.33% at 51.97 ms. However, PaddleOCR outperformed it across both metrics, securing the highest exact match rate (44.67%) at a lower latency (37.92 ms). While a 44.67% exact match rate remains insufficient for production surveillance, PaddleOCR's shortfall stems entirely from a lack of domain-specific fine-tuning rather than an architectural or alphabet mismatch like LPRNet's. PaddleOCR PP-OCRv4 Mobile was therefore selected as the primary recognition candidate, motivating the targeted fine-tuning and optimization detailed in Chapter 4

3.5 Summary

This chapter described the methodological framework underlying the proposed edge-based LPR system. The pipeline was defined as a two-stage, decoupled architecture, with a detection stage operating on 640×640 normalized input and a recognition stage operating on a halved, 160×48 input tensor, supported by a CPU/NPU execution split and a set of targeted optimizations, namely the halved recognition input resolution, aimed at easing later NPU compilation. The datasets used for detection and recognition were characterized, along with the detection, recognition, and hardware metrics used to evaluate the system, distinguishing them from the general metric definitions already presented in Chapter 2. Finally, the baseline model selection process was presented as a desktop GPU benchmarking exercise: Faster R-CNN was excluded on latency grounds, the YOLO nano family was shown to substantially outperform SSD MobileNet v3 in speed, and, once accuracy was measured on the fine-tuned YOLO variants, the candidates were narrowed to two finalists, YOLOv26n and YOLOv8n, with YOLOv11n set aside for trailing YOLOv26n on every accuracy metric but precision. The final choice between the two finalists was left to Chapter 4, to be resolved through further precision- and compatibility-driven testing. On the recognition side, LPRNet's out-of-the-box failure was attributed to a domain shift from its Chinese CCPD pre-training, while PaddleOCR was selected as the recognition candidate on the strength of its superior out-of-the-box accuracy and competitive latency relative to Tesseract OCR and EasyOCR.

4 MODEL OPTIMIZATION FOR KHADAS VIM3

Chapter 3 established the baseline models evaluated for edge deployment, narrowing the detection candidates with two finalists, YOLOv26n and YOLOv8n, and PaddleOCR's PP-OCRv4 Mobile selected for recognition, and benchmarked their unaccelerated performance on a desktop GPU. This chapter details the process by which the detection and recognition models were converted, compiled, and optimized for execution on the VIM3, and describes how the resulting binaries were integrated into a single, hardware-aware end-to-end pipeline. Both models were targeted at the VIM3's dedicated Neural Processing Unit (NPU); as described in Section 4.2, this was achieved for the detection model but not for the recognition model, which was deployed on the CPU instead. Where Chapter 3 focused on which models to use, this chapter addresses how those models were adapted, and in both cases ultimately revised, to meet the specific compiler, precision, and runtime constraints imposed by the target accelerator.

4.1 Target Hardware Description

The target deployment platform for this project is the Khadas VIM3 single-board computer, built around the Amlogic A311D SoC. The A311D exposes a heterogeneous computing architecture composed of two distinct compute domains that are exploited independently throughout this project.

The general-purpose compute domain consists of an ARM CPU cluster combining four higher-performance Cortex-A73 cores clocked at 2.2 GHz with two lower-power Cortex-A53 cores clocked at 1.8 GHz, allowing the operating system to schedule work across cores of differing performance and power characteristics. This cluster executes the host operating system, orchestrates the inference pipeline, and, as detailed in Section 4.3, is responsible for all data-dependent and control-flow-heavy operations that do not map efficiently onto a fixed, static compute graph.

The second compute domain is a dedicated NPU rated at 5.0 TOPS at 8-bit Integer (INT8) precision, with a power draw of 5W. Unlike the CPU cluster, the

NPU is not directly programmable through a general-purpose instruction set; instead, it executes pre-compiled, static computational graphs that are prepared offline through a proprietary toolchain. Access to this hardware is provided through two complementary software components supplied by the board vendor, the Khadas Acuity Toolkit, an offline model conversion and compilation suite that translates a trained network into an NPU-native binary, and the KSNN API, a runtime library used to load these compiled binaries and execute inference requests from within the application code. The characteristics and constraints of this toolchain, rather than the raw specifications of the SoC, are what principally shaped the model conversion process described in the following section, and, in the case of the detection model, ultimately determined which architecture could be deployed at all.

4.2 Model Conversion and Compilation

Neither of the models selected in Chapter 3 could be deployed to the NPU in their original training-time form, and each surfaced a distinct set of compatibility issues that had to be resolved individually. In the case of the detection model, this incompatibility was severe enough to require a change of architecture altogether.

4.2.1 Acuity Toolkit Optimization Methodology

Before presenting the model-specific results, this section describes the general optimization methodology offered by the Khadas Acuity Toolkit, since the same set of parameters is applied to both the detection and recognition models later in this chapter.

Independently of the operator-compatibility constraints discussed in the following sections, the Acuity Toolkit allows a trained model to be compiled at one of three numerical precision formats. 32-bit Floating Point (FP32) compiles the model using its original 32-bit floating-point weights and activations, and serves as an accuracy reference rather than a deployment target, since it does not exploit the NPU's reduced-precision compute paths. FP16 halves the bit-width of weights and activations to 16-bit floating point, reducing memory bandwidth and compute time while preserving the same floating-point number representation, and therefore typically introduces only a small, or no, loss of numerical precision relative to FP32. INT8 goes further, mapping weights and activations onto an 8-bit integer range; because this format targets the NPU's rated 5.0 TOPS throughput figure, it offers the largest theoretical performance ceiling, but it requires the model's activation ranges to be estimated ahead of time from a representative calibration dataset, and this approximation step is a common source of accuracy degradation, particularly for architectures whose activation distributions are not well captured by a small

calibration set.

Independently of precision, the toolkit also supports structured channel pruning, in which a specified proportion of convolutional filters are removed from the trained network prior to compilation, reducing the model's parameter count and computational cost at the risk of degrading its representational capacity if too many filters are removed.

Because precision format and pruning ratio interact, in principle, an aggressively pruned model compiled at a high precision could reach a similar latency to a lightly pruned model compiled at a low precision, but with a different accuracy outcome. Both parameters are therefore evaluated jointly, rather than independently, for each model considered for NPU deployment in this chapter, following pruning and quantization testing principles adapted from NVIDIA TensorRT.

4.2.2 Detector Reselection: From YOLOv26n to YOLOv8n

Preliminary desktop testing, applying the precision and pruning methodology via NVIDIA TensorRT, an inference optimization SDK that compiles a trained network into a hardware-specific engine and applies layer fusion, precision calibration, and structured pruning ahead of deployment on NVIDIA GPU hardware, was carried out on YOLOv26n first, since it was the newer of the two finalist architectures. The results, summarized in Table 4.1, an unpruned FP16 configuration reproducing the FP32 mAP@50 of 99.36% at a reduced inference time of 3.21 ms, indicated that YOLOv26n was, on this desktop-side evidence, the stronger of the two finalists.

Table 4.1: Impact of Pruning and Quantization on YOLOv26n

Pruning Ratio (%)	Precision Format	mAP@50 (%)	Avg Time (ms)
0	FP32	99.36	3.30
0	FP16	99.36	3.21
0	INT8	73.21	3.09
30	FP32	96.07	3.66
30	FP16	96.07	3.26
30	INT8	71.63	3.17
50	FP32	16.50	3.42
50	FP16	16.45	3.54
50	INT8	13.15	3.18

This desktop-side result, however, could not be carried through to the target device. When the trained YOLOv26n model was exported to ONNX and submitted to the Khadas Acuity Toolkit, the conversion process failed. Unsupported dynamic shape errors and operator incompatibilities in the detection head preventing the model from being compiled into an NPU-native binary. Unlike the PaddleOCR incompatibility, which was traced to a single, specifically identified unsupported

activation function and resolved through targeted graph surgery, the Acuity Toolkit provides limited diagnostic output for graph compilation failures of this kind, and Khadas's public documentation does not detail the internal graph validation the compiler performs. Diagnosing this class of failure typically requires iteratively isolating individual subgraphs of the model and resubmitting them to the closed-source compiler, a process that is time-intensive regardless of the operator's familiarity with the toolchain. Given the time constraints of this project, pursuing this diagnostic process to completion was judged to be outside the project's scope, and YOLOv26n was set aside as a deployment candidate in favour of the second finalist.

The detection model was therefore built around YOLOv8n, the nano variant of the earlier, more established YOLOv8 architecture discussed in Section 2.2.2. Although the desktop benchmark in Table 3.1 shows YOLOv8n trailing YOLOv26n by a narrow latency margin, its wider adoption at the time of this project meant that its ONNX export graph was already known to be compatible with the Acuity Toolkit's supported operator set, and it was accepted for compilation without requiring the graph-level modifications that YOLOv26n would have needed. All subsequent conversion, optimization, and pipeline integration work for the detector, described in the remainder of this chapter, was therefore carried out on YOLOv8n rather than YOLOv26n.

4.2.3 YOLOv8n Conversion Pipeline

The YOLOv8n detection model was exported from its native training framework to the ONNX format using a static input shape rather than a dynamic one, since the Acuity Toolkit's graph compiler requires all tensor dimensions to be fully known at compile time in order to allocate the fixed memory buffers used during NPU execution. The resulting static-shape ONNX graph was then passed through the Acuity Toolkit, which compiled it into a pair of NPU-native binary artefacts: a `.nb` file, containing the compiled instruction graph and weights, and an accompanying `.so` shared library, containing the runtime glue code required by KSNN to load and execute the `.nb` binary on the device.

Applying the same precision and pruning methodology described in Section 4.2.1, YOLOv8n was evaluated across the three supported precision formats and three pruning ratios, using the same TensorRT-based desktop methodology, and the same RTX 3060 workstation, used for the YOLOv26n study in Section 4.2.2. Table 4.2 reports the results.

As with Table 4.1, the absolute latency figures in Table 4.2 are desktop GPU measurements and are not representative of on-device NPU latency. They are used here only for the relative comparison between precision formats and pruning ratios. The corresponding on-device latency for the selected configuration, once compiled

Table 4.2: Impact of Pruning and Quantization on YOLOv8n

Pruning Ratio (%)	Precision Format	mAP@50 (%)	Avg Time (ms)
0	FP32	96.79	2.63
0	FP16	96.80	2.88
0	INT8	94.63	2.72
30	FP32	2.00	2.82
30	FP16	2.00	2.79
30	INT8	5.00	2.88
50	FP32	0.00	2.80
50	FP16	0.00	3.12
50	INT8	0.00	2.78

for the Khadas Acuity Toolkit and executed via KSNN, is reported separately in Chapter 5, and is substantially higher than the desktop TensorRT figures below.

At 0% pruning, FP32 and FP16 report near-identical accuracy (96.79% and 96.80% mAP@50 respectively), and average inference time is comparable between the two (2.63 ms and 2.88 ms); unlike the YOLOv26n result in Table 4.1, FP16 does not show a clear average-latency advantage over FP32 for YOLOv8n on this desktop setup. INT8 at 0% pruning retains a substantially higher accuracy (94.63%) than the corresponding YOLOv26n INT8 configuration in Table 4.1 (73.21%), indicating that YOLOv8n's activation distributions are better suited to 8-bit calibration than YOLOv26n's, nonetheless, INT8 still trails FP16 by 2.17 percentage points in accuracy without offering a corresponding latency advantage on this hardware, and was therefore not selected.

Pruning proved far more damaging to YOLOv8n than to YOLOv26n: at 30% pruning, mAP@50 collapses to 2.00-5.00% across all three precision formats, and at 50% pruning the model fails to produce usable detections at any precision (0.00% mAP@50), in contrast to YOLOv26n, which retained 96.07% mAP@50 at 30% pruning and a non-zero 13-16% mAP@50 even at 50% pruning across all precision formats, rather than collapsing to 0.00% entirely.

On the basis of these results, the 0%-pruning, FP16 configuration was selected as the deployment format for YOLOv8n.

4.2.4 PaddleOCR Conversion Pipeline

The recognition model required a more involved conversion process. As a first step, the dependency on the full `paddlepaddle` Python framework, which is not practical to deploy on a resource-constrained ARM target, was eliminated by exporting the trained PP-OCRv4 Mobile model directly to ONNX, allowing all subsequent processing to rely solely on the ONNX ecosystem.

A direct conflict then emerged between the operator support offered by PaddleOCR's native graph and the operator support accepted by the Acuity NPU compiler. The Acuity Toolkit used in this project only accepts graphs expressed under ONNX Operator Set (Opset) version 11 or earlier, whereas PP-OCRv4 Mobile natively relies on the `hard_swish` activation function, an operator only formally introduced in Opset 14. Attempting to compile the unmodified, higher-Opset graph directly resulted in compiler failures, providing no actionable diagnostic information and requiring the incompatibility to be traced manually, operator by operator, back to the activation function.

To resolve this, a custom Python graph surgeon script was developed to manipulate the ONNX computation graph directly at the node level, prior to compilation. This script performed two modifications:

- **Static shape locking**: The model's input tensor shape was fixed to `1x3x48x160`, matching the batch size, channel count, and the halved 160×48 recognition resolution. This step removed residual dynamic-axis metadata left over from the export process, satisfying the same static-shape requirement described for the YOLOv8n in section 4.2.3.

- **Operator decomposition**: Every `hard_swish` node in the graph was replaced with a mathematically equivalent sequence of two Opset-11-compatible nodes. A `HardSigmoid` operation followed by an element-wise `Multiply` operation. Since `hard_swish(x)` is defined as `x` multiplied by a hard-sigmoid function of `x`, this decomposition reproduces the numerical behaviour of the original activation using primitives already available under Opset 11.

Following this graph surgery, the ONNX simplifier tool was applied to strip redundant dynamic-shape nodes and constant-fold the residual "glue"layers left by the export, reducing the graph to the minimal operators required for inference.

4.2.5 PaddleOCR NPU Compilation Failure and CPU Fallback

With the `hard\_swish` operator resolved and the graph reduced to Opset 11, the simplified PaddleOCR graph was submitted to the Acuity Toolkit for compilation into a `.nb` binary. Despite satisfying the Opset and static-shape requirements established in Section 4.2.4, compilation still failed: the compiler process terminated abruptly with a generic "Internal Compiler Error"during the graph translation phase, yielding no detailed stack trace or specific error log. Unlike the YOLOv26n case described in Section 4.2.2, where the failure was traced to identifiable causes in the detection head, this second PaddleOCR failure could not be attributed to a

specific operator or graph pattern from the Acuity Toolkit's diagnostic output alone. A plausible explanation, however, follows from the composition of the PP-OCRv4 Mobile recognition head: unlike YOLOv8n's purely convolutional backbone, which compiled without incident, PP-OCRv4 Mobile's Single Visual model for Text Recognition (SVTR)-based architecture combines a lightweight CNN backbone with a small number of transformer-style sequence-encoder blocks ahead of the CTC output layer. Embedded NPU toolchains, including the Acuity Toolkit, are typically optimized first for the convolutional operator patterns common to detection backbones, and comparatively less mature for the layer-normalization and sequence-reshaping operations a transformer-based encoder requires, making this class of architecture a more likely source of unsupported graph patterns than a purely convolutional one. Fully confirming this would have required the same time-intensive, iterative subgraph isolation process considered and set aside for YOLOv26n, which was judged to be outside the time available for this project; the resulting CPU deployment, described below, was therefore a deliberate engineering decision made under this constraint, rather than a default fallback.

Consequently, PaddleOCR PP-OCRv4 Mobile was not deployed to the NPU. Instead, the Opset-11-compliant ONNX graph produced in Section 4.2.4 was deployed directly on the ARM CPU cluster using ONNX Runtime. Unlike YOLOv8n, which executes as a compiled `.nb` binary through KSNN, PaddleOCR therefore runs as an interpreted ONNX graph on the CPU for all results reported from this point onward, including the end-to-end pipeline described in Section 4.3 and the experimental results reported in Chapter 5.

4.2.6 PaddleOCR Optimization Ablation

With the graph compiling successfully, a set of optimization phases was applied incrementally and measured against the baseline model to determine their individual and combined contribution to recognition performance. Table 4.3 reports the results.

Table 4.3: Ablation Study of PaddleOCR Optimization Techniques

Optimization Phase	**Exact Match (%)**	**Char Sim. (%)**	**Avg Time (ms)**
Baseline PaddleOCR	66.41	87.95	37.92
Fine-Tuning	72.60	91.80	34.13
Syntactic Correction	84.99	96.35	45.86
FP16 Quantization	**84.85**	**96.33**	**3.79**

Fine-tuning the model on the domain-specific dataset described in Section 3.2 raises exact match accuracy from 66.41% to 72.60% and character similarity from 87.95% to 91.80%, while also reducing average inference time from 37.92 ms to 34.13 ms. Applying the syntactic correction step described in Section 3.1 on top of

the fine-tuned model raises exact match accuracy further to 84.99% and character similarity to 96.35%, at the cost of increased average time (45.86 ms), attributable to the added regex-based post-processing pass. FP16 quantization, measured via NVIDIA TensorRT on the desktop GPU, reduces average inference time from 45.86 ms to 3.79 ms, a reduction of 42.07 ms, while exact match accuracy falls by 0.14 percentage points (84.99% to 84.85%) and character similarity falls by 0.02 percentage points (96.35% to 96.33%). Given that this quantization step accounts for the overwhelming majority of the total latency reduction achieved across all four phases, while degrading accuracy by less than 0.2 percentage points, FP16 was retained as the deployment precision for the recognition model, consistent with the precision selected for detection in Section 4.2.3.

4.3 End-to-End Pipeline Design

The compiled NPU binaries produced in Section 4.2 are not deployed as standalone inference engines. Instead, they form part of a broader end-to-end pipeline that distributes computation between the NPU and the ARM CPU cluster, assigning each stage to the compute domain best suited to its computational profile. Following the revision described in Section 4.2.2, the detection component of this pipeline is YOLOv8n rather than YOLOv26n. Figure 4.1 illustrates this hardware/software distribution across the full pipeline.

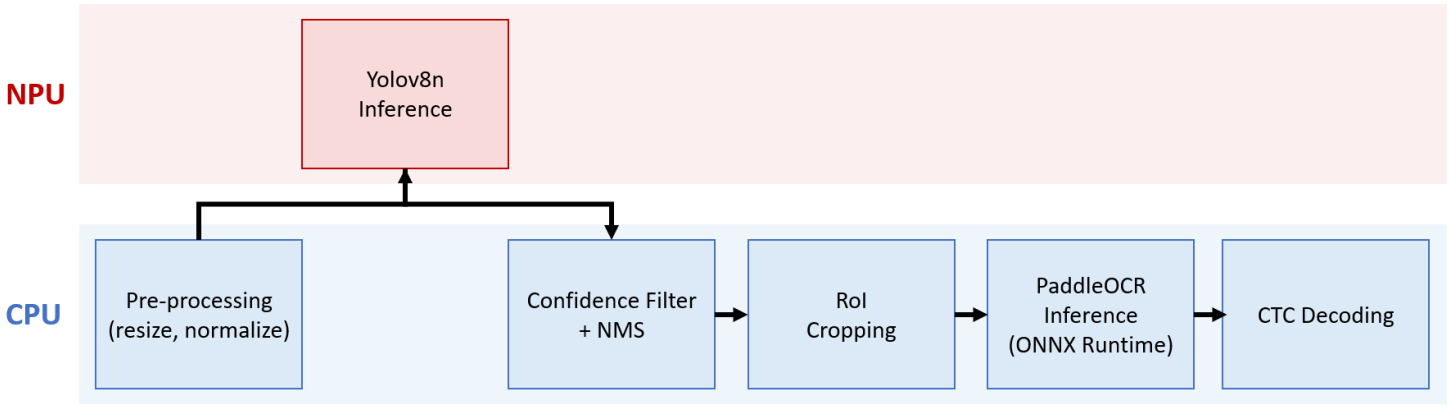


Figure 4.1: *Hardware and Software Partitioning Across Khadas VIM3 Heterogeneous Compute Domains*

4.3.1 NPU: Hardware-Accelerated Feature Extraction

The NPU, accessed through the KSNN runtime, is reserved exclusively for the convolutional feature extraction of YOLOv8n. Because this network was compiled ahead-of-time into a fixed-shape, FP16 `.nb` binary, as described in Section 4.2.3, the KSNN engine executes its forward pass as a single, predictable graph invocation, without runtime branching or dynamically sized intermediate tensors. This choice is validated empirically in Section 5.2.1, where YOLOv8n measured on the VIM3 CPU via ONNX Runtime is compared directly against the same model on the NPU, showing an approximately 3.0× reduction in inference latency from NPU offloading alone. PaddleOCR does not run on the NPU in this pipeline. As established in Section 4.2.5, its ONNX graph could not be compiled into a `.nb` binary, and its convolutional backbone is therefore executed on the CPU alongside the pipeline's other CPU-resident stages, described below.

4.3.2 CPU: Control Flow and Post-Processing

After the NPU returns the three-scale feature maps for YOLOv8n, the ARM CPU cluster takes over for the remainder of the pipeline, including the entire recognition model. Operations involving variable-length loops, sorting algorithms, or data-dependent branching cannot be efficiently expressed as static compute graphs and are therefore strictly assigned to the CPU.

On the detection side, this CPU-resident stage covers the full detection post-processing pipeline: decoding raw feature-map outputs into candidate bounding box coordinates, applying the confidence threshold, performing NMS to eliminate duplicate detections, and cropping the resulting Region of Interest from the source image, as described in Section 3.1. Each of these steps involves a variable number of candidate boxes per frame, making them poorly suited to a fixed-size NPU graph but comparatively inexpensive to execute on a general-purpose CPU core.

The cropped RoI is subsequently passed to PaddleOCR, which runs as an ONNX graph via ONNX Runtime on the CPU. The CPU then processes the output using the greedy Numpy-based CTC decoder (Section 3.1). This routine sequentially iterates through the timesteps, merges consecutive identical predictions, and removes the CTC blank token. One critical adjustment was required during integration, the PaddleOCR blank token had to be explicitly mapped to index 0 of the character dictionary. Without this mapping, the decoded characters shifted by one position, rendering the output completely illegible despite the underlying model inference being numerically correct. Once corrected, the system produced coherent, accurately aligned plate strings.

4.3.3 Combined Pipeline Flow

In the finalized end-to-end pipeline, only the detection model leverages NPU acceleration (Figure 4.1). Initially, the CPU resizes and normalizes the input frame. The NPU then executes the YOLOv8n forward pass. Following this, the CPU resumes control to perform detection post-processing, RoI cropping, PaddleOCR inference, CTC decoding, and final syntactic text filtering. This distribution diverges from the original methodology proposed in Section 4.2, which intended to accelerate both models on the NPU. This revision is a direct consequence of the PaddleOCR compilation failure (Section 4.2.5), meaning the recognition stage operates without the substantial FP16 acceleration that was observed on the desktop GPU via TensorRT (Section 4.2.6). The exact latency cost of each stage including the CPU bound recognition inference is detailed in Chapter 5 using the per-stage breakdown defined in Section 3.3.

4.4 Summary

This chapter described the conversion, compilation, and precision optimization of the detection and recognition models for the Khadas VIM3, and the two points at which the intended deployment plan had to be revised in response to Acuity Toolkit compilation failures. The VIM3's heterogeneous A311D SoC, combining an ARM Cortex-A73/A53 CPU cluster with a 5.0 TOPS NPU accessed through the Acuity Toolkit and KSNN API, was described as the target platform, along with the toolkit's general precision and pruning optimization methodology.

Chapter 3 narrowed the detection candidates to two finalists, YOLOv26n and YOLOv8n. Preliminary desktop benchmarking using TensorRT favoured YOLOv26n, and a pruning and precision study on this model (Table 4.1) selected FP16 at 0% pruning as the strongest configuration; however, YOLOv26n's detection head combined residual dynamic-shape metadata with a small number of operators outside the Acuity Toolkit's supported Opset 11 set, and resolving this combination was judged too time-intensive within the scope of this project, unlike the single-operator fix that resolved PaddleOCR's `hard_swish` incompatibility. The detector was therefore built around YOLOv8n instead, which compiled successfully into `.nb`/`.so` NPU binaries and was carried forward through the remainder of the pipeline.

PP-OCRv4 Mobile required a custom graph surgeon script to lock its input shape and decompose its `hard_swish` activation into Opset-11-compatible operations, resolving the Opset 14 incompatibility of the original graph. A subsequent desktop ablation study using TensorRT (Table 4.3) showed that fine-tuning, syntactic correction, and FP16 quantization combined could, in principle, raise exact match accuracy from 66.41% to 84.85% while reducing average inference time from 37.92

ms to 3.79 ms. Despite resolving the `hard_swish` incompatibility, however, the Opset-11-compliant PaddleOCR graph still failed to compile into a `.nb` binary for the Acuity Toolkit, for reasons distinct from the `hard_swish` issue and, as with YOLOv26n, impractical to resolve within the time available. PaddleOCR was therefore deployed on the CPU via ONNX Runtime rather than on the NPU, meaning the 3.79 ms FP16 figure was not achieved on the target device.

The resulting end-to-end pipeline accelerates only the detection stage on the NPU, detection post-processing, RoI cropping, PaddleOCR recognition inference, and CTC decoding all execute on the CPU, with the blank-token indexing fix identified as a critical correction for producing coherent recognition output. This pipeline, and its measured accuracy and latency characteristics relative to the desktop baselines established in Chapter 3, are evaluated in full in Chapter 5.

5 EXPERIMENTAL EVALUATION AND RESULTS

5.1 Experimental Setup

Evaluation was performed on the Khadas VIM3. The detection model, YOLOv8n, was compiled to a .nb NPU binary and executed on the 5.0 TOPS NPU via the KSNN API

Evaluation was performed on the Khadas VIM3 using two held-out test sets: an 800-image set for detection and a 734-image set for recognition. The detection model, YOLOv8n, was compiled to a .nb NPU binary and executed on the 5.0 TOPS NPU via the KSNN API. The recognition model, PaddleOCR PP-OCRv4 Mobile, was kept in ONNX format and executed on the ARM CPU cluster via ONNX Runtime. All non-inference pipeline stages, including confidence filtering, NMS, RoI cropping, and CTC decoding, were executed on the CPU, as described with the workload allocation in Chapter 4.

5.2 System Performance

This section reports the measured accuracy and latency of the detection stage, the recognition stage, and the full end-to-end pipeline, each evaluated independently on the VIM3.

5.2.1 Object Detection Performance

YOLOv8n was evaluated on the full 800-image detection test set described in Section 5.1, at a confidence threshold of 0.60 and an NMS IoU threshold of 0.50, with inference executed on the NPU and post-processing (confidence filtering and NMS) executed on the CPU. Table 5.1 reports the results, with latency given as the average measured across all 800 images.

At the 0.60 confidence threshold, the model produces 938 detections against 1000 ground-truth labels, a detection count 6.2% below the label count, consistent with the 86.30% recall figure. Precision (92.00%) exceeds recall, indicating that false positives are comparatively less frequent than missed detections at this threshold.

Table 5.1: YOLOv8n Object Detection Performance Metrics

Metric	Result
mAP@50 (%)	82.41
Precision (%)	92.00
Recall (%)	86.30
Latency (ms)	107.6
Total Images	800
Total Labels	1000
Total Detections	938

The reported latency of 107.6 ms in Table 5.1 reflects the full detection pipeline, representing the sum of NPU inference and CPU post-processing (confidence filtering and NMS), excluding initial image pre-processing. the isolated NPU-only inference time is reported separately as part of the per-stage breakdown in Section 5.2.3. This on-device latency is substantially higher than the 18.61 ms measured for YOLOv8n on the desktop RTX 3060 GPU (Table 3.1), a difference attributable to the lower raw compute throughput of the 5.0 TOPS NPU relative to a discrete desktop GPU. While the desktop TensorRT evaluation (Table 4.2) confirmed FP16 as the optimal precision format for YOLOv8n, a corresponding precision-format ablation was not performed directly on the VIM3 NPU.

To isolate the contribution of the NPU itself, YOLOv8n was additionally run directly on the VIM3's ARM CPU cluster via ONNX Runtime, using the same input pipeline and test conditions as the NPU run reported above. Table 5.2 reports the comparison.

Table 5.2: Latency Comparison Between CPU and NPU Execution

Compute Domain	Avg Latency (ms)	Min Latency (ms)	Max Latency (ms)
CPU (ONNX Runtime)	329.6	323.6	391.8
NPU (KSNN)	110.1	107.7	119.4

NPU execution reduces average YOLOv8n inference latency from 329.6 ms to 110.1 ms, a speedup of approximately 3.0×. This result confirms, on the target device itself, the benefit of NPU offloading assumed in the pipeline design of Section 4.3: without hardware acceleration, detection alone would consume 329.6 ms per frame, more than the total end-to-end pipeline latency measured with the NPU in use, reported in Section 5.2.3. The NPU-run average (110.1 ms) is consistent with, though not identical to, the 107.6 ms figure reported earlier in this section; the 2.5 ms difference falls within the min-max range observed across both runs (107.7-119.4 ms) and reflects normal run-to-run variance rather than a discrepancy in the underlying

measurement.

This on-device NPU latency (110.1 ms) remains substantially higher than the deployed fine-tuned model's desktop GPU latency reported in Table 3.2 (10.77 ms), a gap of approximately 10×. Taken together, the three measurements form a clear hierarchy: the discrete desktop GPU (10.77 ms) is fastest, unaccelerated on-device CPU execution (329.6 ms) is slowest, and the NPU (110.1 ms) recovers roughly two-thirds of that gap without matching discrete-GPU performance. This confirms that NPU offloading is worthwhile on this hardware, while also indicating that the 5.0 TOPS NPU does not fully substitute for a discrete GPU's raw compute throughput, a limitation consistent with the accelerator's embedded power and area budget.

5.2.2 Character Recognition Performance

PaddleOCR PP-OCRv4 Mobile was evaluated on the 734-image recognition test set. The recognition model was executed in its ONNX form on the CPU via ONNX Runtime rather than compiled to the NPU. Table 5.3 reports the results.

Table 5.3: PaddleOCR Recognition Performance Metrics

Metric	**Result**
Exact Match (%)	89.65
Character Similarity (%)	98.06
Character Error Rate (CER, %)	1.94
Latency (ms)	48.2

The on-device exact match accuracy (89.65%) exceeds the desktop Syntactic Correction figure reported in Table 4.3 (84.99%), measured on a different held-out set, by 4.66 percentage points, and the desktop TensorRT FP16 figure (84.85%) by 4.80 percentage points. The on-device measured latency (48.2 ms on the VIM3 CPU) is slightly higher than the 45.86 ms Syntactic Correction figure reported in Table 4.3. Because the Table 4.3 ablation was measured on the desktop workstation rather than the edge device, this 48.2 ms result demonstrates that the VIM3's ARM cluster successfully maintains highly competitive inference speeds even without desktop-class processors. The 3.79 ms figure recorded under TensorRT-based FP16 quantization was not reproduced on-device, since, as established in Section 4.2.5, PaddleOCR could not be compiled to the VIM3 NPU and therefore did not benefit from hardware acceleration in this deployment.

5.2.3 Full System Performance

The full pipeline was evaluated end-to-end, decomposing total latency by stage as defined in Section 3.3. Table 5.4 reports the per-stage latency breakdown.

Pipeline Stage	Compute Domain	Latency (ms)	Share of Total (%)
Pre-processing (resize, normalize)	CPU	9.97	5.9
YOLOv8n Inference	NPU	91.59	54.5
CPU Post-processing (confidence filter + NMS)	CPU	17.58	10.5
RoI Cropping	CPU	0.62	0.4
PaddleOCR Inference (ONNX Runtime)	CPU	48.06	28.6
CTC Decoding	CPU	0.14	0.1
Total End-to-End Latency	—	**167.96**	**100**

Table 5.4: End-to-End Pipeline Latency Breakdown

YOLOv8n NPU inference is the single largest contributor to total latency, accounting for 54.5% of the end-to-end time despite running on the hardware accelerator. PaddleOCR CPU inference is the second-largest contributor at 28.6%. Combined, the two model inference stages account for 83.1% of total latency, while the remaining CPU-bound control-flow stages, pre-processing, NMS, RoI cropping, and CTC decoding, together contribute 28.31 ms, or 16.9% of the total. Of these control-flow stages, CPU post-processing (confidence filtering and NMS) is the largest single contributor at 17.58 ms, consistent with its role in filtering and de-duplicating a variable number of candidate boxes per frame, as discussed in Section 4.3.2. RoI cropping and CTC decoding are negligible, at 0.62 ms and 0.14 ms respectively.

Based on the total end-to-end latency of 167.96 ms reported in Table 5.4, the system achieves a throughput of **5.95 FPS**.

5.3 Qualitative Results

In addition to the quantitative metrics reported above, this section presents representative example frames to illustrate the pipeline's end-to-end behaviour qualitatively.



Figure 5.1: *Example of a captured vehicle frame showing standard European plate proportions. Recognized text: "AA01AA"(ground truth: "AA 01 AA").*



Figure 5.2: *Example of a close-up rear capture of a license plate. Recognized text: "6900MA"(ground truth: "69 00 MA")*

Figures 5.1 and 5.2 show two representative success cases, captured from different viewing angles and lighting conditions. In both, the detector correctly localizes the plate, and the cropped RoI is transcribed by PaddleOCR with an exact character match against the ground truth.



Figure 5.3: *Example of a synthetic vehicle image showing a rear-mounted license plate. Recognized text: "N88"(ground truth: "NOH E 8843")*

Figure 5.3 illustrates a representative failure case, in which the recognized output ("N88") retains only a small fraction of the ground-truth sequence ("NOH E 8843"). The detection bounding box correctly covers the full plate, ruling out an incomplete RoI crop as the cause. The failure therefore originates entirely within the recognition stage. Specifically, the physical plate features a two-line layout ("NOH E"above "8843"), whereas the recognizer's fixed 160x48 input tensor is heavily optimized for standard single-line European plates. Forcing a stacked, two-line crop into this elongated aspect ratio severely distorts the character geometries, which likely causes the CTC decoding to collapse most of the sequence rather than misreading individual characters. Unlike the character-level substitutions the syntactic correction step described in Section 3.1 is designed to catch (for example, '0' versus 'O', '8' versus 'B'), this near-total sequence loss cannot be repaired by a step that only corrects individual misclassified characters within an otherwise complete sequence, and represents a more fundamental recognition-stage limitation, motivating further robustness work of the kind discussed in Section 6.2.

5.4 Comparison to Related Work

Table 5.5 compares the proposed pipeline against reference figures for traditional two-stage detection and general-purpose OCR, drawn from the desktop baseline benchmarks in Chapter 3 and the literature reviewed in Chapter 2.

System	Detector	Detection Latency (ms)	Recognizer	Recognition Accuracy	Recognition Latency (ms)	Target Hardware
Proposed Pipeline	YOLOv8n (NPU)	107.6	PaddleOCR PP-OCRv4 Mobile (CPU, ONNX Runtime)	89.65% (Exact Match)	48.2	Khadas VIM3 (5.0 TOPS NPU + ARM CPU)
Traditional Two-Stage (Faster R-CNN)	Faster R-CNN	4573.80	—	—	—	Desktop GPU (RTX 3060)
General-Purpose OCR (Tesseract, out-of-the-box)	—	—	Tesseract OCR	5.33% (Exact Match)	129.99	Desktop GPU (RTX 3060)
Literature Baseline (Tykva [33])	Faster R-CNN	71.0	Custom HTR	0.58% (CER)	23.65	Tesla P100 (detection) / NVIDIA 930MX (recognition)

Table 5.5: Comparative Performance Evaluation Against Baselines and Literature

The proposed detection stage (107.6 ms) is substantially faster than the desktop-measured Faster R-CNN baseline (4573.80 ms) by more than 40×, despite the latter running on a discrete desktop GPU (RTX 3060) rather than an embedded NPU. This confirms that the single-stage versus two-stage latency gap established in Section 3.4.1 holds even after accounting for the move to constrained hardware. On the recognition side, the proposed pipeline's exact match accuracy (89.65%) exceeds the out-of-the-box Tesseract OCR baseline (5.33%) by a wide margin, and its CPU-only latency (48.2 ms) is lower than Tesseract's desktop latency (129.99 ms), indicating that the fine-tuned, syntactically corrected PaddleOCR model outperforms a generic, non-domain-specific OCR engine on both accuracy and speed even without hardware acceleration.

Furthermore, when compared against the literature baseline established by Tykva [33], the comparison must account for the substantial difference in target hardware. Tykva's Faster R-CNN detector, evaluated on an NVIDIA Tesla P100 GPU via Google Colaboratory, a full server-class accelerator, achieves a detection latency of approximately 71 ms (14 FPS) and a detection accuracy of 98.15% AP. The proposed

YOLOv8n detector, at 107.6 ms on the VIM3's 5.0 TOPS embedded NPU, trails this figure in raw latency, but does so on a device drawing a fraction of the power and physical footprint of a server-class GPU, a distinction the latency figures alone do not capture. On the recognition side, Tykva's Custom Handwritten Text Recognition (HTR) engine, evaluated on a local NVIDIA GeForce 930MX GPU, achieves a lower recognition latency (23.65 ms) and an excellent Character Error Rate (0.58%); the proposed pipeline's PaddleOCR-based recognizer, executing entirely on a mobile-grade ARM CPU cluster with no GPU acceleration at all, remains competitive at 48.2 ms, a reasonable outcome given the complete absence of a dedicated accelerator for this stage.

5.5 Summary

This chapter reported the measured performance of the deployed pipeline on the Khadas VIM3. The detection model, YOLOv8n, executed on the NPU, achieved an mAP@50 of 82.41%, a precision of 92.00%, and a recall of 86.30%, at a latency of 107.6 ms. The recognition model, PaddleOCR PP-OCRv4 Mobile, executed on the CPU via ONNX Runtime after NPU compilation proved unsuccessful (Section 4.2.5), achieved an exact match accuracy of 89.65%, a character similarity of 98.06%, and a CER of 1.94%, at a latency of 48.2 ms. The full end-to-end pipeline, decomposed across pre-processing (9.97 ms), detection inference (91.59 ms), detection post-processing (17.58 ms), RoI cropping (0.62 ms), recognition inference (48.06 ms), and CTC decoding (0.14 ms), produced a total latency of 167.96 ms, corresponding to a throughput of 5.95 FPS. Detection inference and recognition inference together account for 83.1% of total latency, with detection inference alone the single largest contributor despite running on the NPU. Compared against the reference figures established in Chapter 3, the proposed detector substantially outperforms Faster R-CNN in latency, and the proposed recognizer substantially outperforms out-of-the-box Tesseract OCR in both accuracy and latency; a quantitative comparison against Jetson-class GPU-accelerated systems remains limited by the granularity of the throughput figures reported in the literature. These results are discussed in the context of the thesis objectives, and directions for future work, in Chapter 6.

6 CONCLUSION AND FUTURE WORK

6.1 Conclusion

The primary objective of this project was to design, evaluate, and implement a lightweight, deep learning-based ALPR pipeline optimized for real-time inference on resource-constrained edge devices. As established in Chapter 1, deploying state-of-the-art computer vision models at the edge introduces strict constraints regarding latency, computational resources, and hardware compatibility. To qualify as a viable solution for real-time traffic surveillance, the system was required to meet a defined set of empirical performance targets: a pipeline inference latency of no more than 200 ms per frame (corresponding to a minimum throughput of 5 FPS), a detection accuracy of at least 75% mAP@50, and a recognition accuracy of at least 75% exact-match.

Through a progressive methodology of desktop benchmarking, graph surgery, precision quantization, and heterogeneous hardware integration, this thesis successfully realized a two-stage pipeline on the Khadas VIM3 single-board computer. The final architecture distributed the computational workload across the Amlogic A311D System-on-Chip by accelerating the YOLOv8n detection model on the 5.0 TOPS Neural Processing Unit (NPU) at FP16 precision, while delegating the PaddleOCR PP-OCRv4 Mobile recognition engine and all data-dependent post-processing to the ARM CPU cluster.

The experimental evaluation detailed in Chapter 5 confirms that the proposed pipeline successfully exceeds all foundational project objectives:

- **Detection Performance:** The YOLOv8n detector achieved an 82.41% mAP@50 and a 92.00% precision rate at an operational confidence threshold of 0.60, comfortably exceeding the 75% baseline target.
- **Recognition Performance:** The domain-fine-tuned and syntactically corrected PaddleOCR engine achieved an 89.65% exact-sequence match rate and a 1.94% CER, demonstrating robust character legibility that surpasses the 75% accuracy objective.

- **System Latency and Throughput:** End-to-end execution of the full pipeline required 167.96 ms per frame, equating to a throughput of 5.95 FPS. By successfully operating beneath the 200 ms real-time threshold, the system ensures that vehicles can be reliably identified in dynamic surveillance scenarios without introducing unmanageable frame processing backlogs.

Ultimately, this dissertation demonstrates that the limitations of traditional, computationally heavy multi-stage detectors and general-purpose OCR engines can be overcome at the edge. By leveraging hardware-aware precision quantization and hybrid CPU/NPU execution, high-accuracy ALPR can be deployed autonomously on resource-constrained, affordable hardware, effectively removing the dependency on continuous cloud connectivity for intelligent traffic monitoring.

6.2 Future Work

While the proposed pipeline meets all defined objectives, the technical constraints encountered during implementation suggest several avenues for future research and optimization:

- **NPU Compiler Resolution for Advanced Architectures:** As detailed in Chapter 4, the newer YOLOv26n model demonstrated superior desktop accuracy but failed compilation on the Khadas Acuity Toolkit due to unsupported dynamic shapes and operator incompatibilities in the detection head. Future work should focus on reverse-engineering or adapting the NPU compiler to accept these newer, more efficient architectures, which could further increase detection precision.

- **Full Pipeline Hardware Acceleration:** Currently, the PaddleOCR engine executes entirely on the ARM CPU cluster (consuming 48.06 ms per frame) because its ONNX graph could not be successfully targeted to the NPU. Future research should aim to port the OCR convolutional backbone directly to the NPU. Migrating this heavy feature-extraction workload off the CPU would reduce recognition latency and improve the total system throughput, bringing it closer to the pipeline's theoretical upper bound of approximately 8.3 FPS.

- **Integration of Object Tracking:** To maximize edge throughput, a lightweight kinematic tracking algorithm (such as DeepSORT or ByteTrack) could be integrated into the pipeline. Rather than running the computationally expensive YOLO detector on every single frame, the system could run detection intermittently, relying on the low-latency tracker to follow the vehicle across subsequent frames.

- **Harsh Environment Dataset Expansion:** The system performs reliably under typical conditions, but expanding the training dataset to include extreme weather conditions (such as heavy rain, fog) and infrared night-vision captures would further harden the models. This would ensure the pipeline remains highly resilient for unconstrained, 24/7 deployment in remote or rural areas.

Bibliography

[1] J. Andersson. "A Study on Automatic License Plate Recognition". Bachelor's thesis. Turku, Finland: Abo Akademi University, 2022. URL: https://urn.fi/URN:NBN:fi-fe2022052638911 (cit. on p. 12).

[2] Y. Cang and S. Lan. "Design of an FPGA-Based High-Efficiency YOLO Accelerator". In: *2025 5th International Conference on Electronics, Circuits and Information Engineering (ECIE)*. 2025, pp. 655–661. DOI: 10.1109/ECIE65947.2025.11086970 (cit. on pp. 1, 15).

[3] C. Cui, T. Sun, M. Lin, T. Gao, Y. Zhang, J. Liu, X. Wang, Z. Zhang, C. Zhou, H. Liu, Y. Zhang, W. Lv, K. Huang, Y. Zhang, J. Zhang, J. Zhang, Y. Liu, D. Yu, and Y. Ma. *PaddleOCR 3.0 Technical Report*. 2025. arXiv: 2507.05595 [cs.CV]. URL: https://arxiv.org/abs/2507.05595 (cit. on p. 14).

[4] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. arXiv: 2010.11929 [cs.CV]. URL: https://arxiv.org/abs/2010.11929 (cit. on p. 8).

[5] S. Ferreira and M. Véstias. "On-Device Real-Time License Plate Identification for Surveillance Systems". In: *Young Engineers Forum on Electrical and Computer Engineering (YEF-ECE)*. 2026 (cit. on p. 2).

[6] S. Ferreira and M. Véstias. "Real-Time License Plate Recognition on Embedded Devices for Surveillance Applications". In: *Simpósio de Informática (INForum)*. 2026 (cit. on p. 3).

[7] J. Fowers, G. Brown, P. Cooke, and G. Stitt. "A performance and energy comparison of FPGAs, GPUs, and multicores for sliding-window applications". In: *Symposium on Field Programmable Gate Arrays*. 2012. URL: https://api.semanticscholar.org/CorpusID:11115837 (cit. on p. 15).

[8] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber. "Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks". In: *Proceedings of the 23rd International Conference on*

Machine Learning. ICML '06. Pittsburgh, Pennsylvania, USA: Association for Computing Machinery, 2006, 369–376. ISBN: 1595933832. DOI: 10.1145/1143844.1143891. URL: https://doi.org/10.1145/1143844.1143891 (cit. on p. 9).

[9] N. S. Hashemi, R. B. Aghdam, A. S. B. Ghiasi, and P. Fatemi. *Template Matching Advances and Applications in Image Analysis*. 2016. arXiv: 1610.07231 [cs.CV]. URL: https://arxiv.org/abs/1610.07231 (cit. on p. 12).

[10] K. He, X. Zhang, S. Ren, and J. Sun. *Deep Residual Learning for Image Recognition*. 2015. arXiv: 1512.03385 [cs.CV]. URL: https://arxiv.org/abs/1512.03385 (cit. on p. 13).

[11] JaidedAI. *EasyOCR*. https://github.com/JaidedAI/EasyOCR. 2020 (cit. on p. 14).

[12] G. Jocher, A. Chaurasia, and J. Qiu. *Ultralytics YOLOv8*. Version 8.0.0. 2023. URL: https://github.com/ultralytics/ultralytics (cit. on pp. 13, 14).

[13] G. Jocher and J. Qiu. *Ultralytics YOLO11*. Version 11.0.0. 2024. URL: https://github.com/ultralytics/ultralytics (cit. on p. 13).

[14] Z. Li, F. Z. Hong, and C. P. Yue. *FPGA-based Acceleration of Neural Network for Image Classification using Vitis AI*. 2024. arXiv: 2412.20974 [cs.CV]. URL: https://arxiv.org/abs/2412.20974 (cit. on p. 10).

[15] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg. *SSD: Single Shot MultiBox Detector*. 2016. DOI: 10.1007/978-3-319-46448-0_2. URL: http://dx.doi.org/10.1007/978-3-319-46448-0_2 (cit. on p. 9).

[16] luigi-ga. *ALPRNet: Automatic License Plate Recognition Network*. https://github.com/luigi-ga/ALPRNet. Accessed: 2026-01-20. n.d. (Cit. on p. 14).

[17] D. Meimetis, I. Daramouskas, N. Patrinopoulou, V. Lappas, and V. Kostopoulos. "Comparative Analysis of Object Detection Models for Edge Devices in UAV Swarms". In: *Machines* 13.8 (2025). ISSN: 2075-1702. DOI: 10.3390/machines13080684. URL: https://www.mdpi.com/2075-1702/13/8/684 (cit. on pp. 13, 15).

[18] NVIDIA Corporation. *Embedded Systems for Autonomous Machines*. Accessed: 2026-01-20. 2026. URL: https://www.nvidia.com/en-gb/autonomous-machines/embedded-systems/ (cit. on p. 15).

[19] N. Omar, S. Zeebaree, M. M.Sadeeq, R. Zebari, H. Shukur, A. Alkhayyat, L. Haji, and S. Kak. *License plate detection and recognition: A study of review*. Jan. 2023. DOI: 10.1063/5.0170932 (cit. on p. 12).

[20] H. Padmasiri, J. Shashirangana, D. Meedeniya, O. Rana, and C. Perera. "Automated License Plate Recognition for Resource-Constrained Environments". In: *Sensors* 22.4 (2022). ISSN: 1424-8220. URL: https://www.mdpi.com/1424-8220/22/4/1434 (cit. on p. 1).

[21] J. Pedoeem and R. Huang. *YOLO-LITE: A Real-Time Object Detection Algorithm Optimized for Non-GPU Computers*. 2018. arXiv: 1811.05588 [cs.CV]. URL: https://arxiv.org/abs/1811.05588 (cit. on p. 17).

[22] R. Prokscha, M. Herrmann, and A. Hoess. *Efficient Edge Deployment Demonstrated on YOLOv5 and Coral Edge TPU*. Aug. 2023. DOI: 10.1201/9781003377382-11 (cit. on p. 10).

[23] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. *You Only Look Once: Unified, Real-Time Object Detection*. 2016. arXiv: 1506.02640 [cs.CV]. URL: https://arxiv.org/abs/1506.02640 (cit. on p. 8).

[24] J. Redmon and A. Farhadi. *YOLOv3: An Incremental Improvement*. 2018. arXiv: 1804.02767 [cs.CV]. URL: https://arxiv.org/abs/1804.02767 (cit. on p. 13).

[25] S. Ren, K. He, R. Girshick, and J. Sun. *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. 2016. arXiv: 1506.01497 [cs.CV]. URL: https://arxiv.org/abs/1506.01497 (cit. on p. 8).

[26] K. Seshadri, B. Akin, J. Laudon, R. Narayanaswami, and A. Yazdanbakhsh. *An Evaluation of Edge TPU Accelerators for Convolutional Neural Networks*. 2022. arXiv: 2102.10423 [cs.LG]. URL: https://arxiv.org/abs/2102.10423 (cit. on p. 10).

[27] J. Shashirangana, H. Padmasiri, D. Meedeniya, C. Perera, S. R. Nayak, J. Nayak, S. Vimal, and S. Kadry. "License plate recognition using neural architecture search for edge devices". In: *International Journal of Intelligent Systems* 37.12 (2022), pp. 10211–10248. DOI: https://doi.org/10.1002/int.22471. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/int.22471. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/int.22471 (cit. on p. 1).

[28] B. Shi, X. Bai, and C. Yao. *An End-to-End Trainable Neural Network for Image-based Sequence Recognition and Its Application to Scene Text Recognition*. 2015. arXiv: 1507.05717 [cs.CV]. URL: https://arxiv.org/abs/1507.05717 (cit. on p. 13).

[29] R. Smith. "An Overview of the Tesseract OCR Engine". In: *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*. Vol. 2. 2007, pp. 629–633. DOI: 10.1109/ICDAR.2007.4376991 (cit. on pp. 13, 17).

[30] T. P. Swaminathan, C. Silver, and T. Akilan. *Benchmarking Deep Learning Models on NVIDIA Jetson Nano for Real-Time Systems: An Empirical Investigation*. 2024. arXiv: 2406.17749 [cs.AR]. URL: https://arxiv.org/abs/2406.17749 (cit. on p. 15).

[31] J. Tang, X. Zheng, Q. Wu, and J. Cui. "A Real-time UAV Object Detection System Design with FPGA Implementation". In: *Proceedings of the 8th International Conference on Computing and Artificial Intelligence*. ICCAI '22. Tianjin, China: Association for Computing Machinery, 2022, 472–477. ISBN: 9781450396110. DOI: 10.1145/3532213.3532284. URL: https://doi.org/10.1145/3532213.3532284 (cit. on p. 15).

[32] M. Tiele. *European License Plate Dataset*. https://www.kaggle.com/datasets/tielemarvin/european-license-plate-dataset-elpd. Accessed: 2026-07-15 (cit. on p. 22).

[33] J. Tykva. *Detection and Recognition of License Plates*. Bachelor's thesis. Brno, CZ, 2020. URL: https://www.fit.vut.cz/study/thesis/23227/ (cit. on pp. 4, 12, 14, 45).

[34] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding. *YOLOv10: Real-Time End-to-End Object Detection*. 2024. arXiv: 2405.14458 [cs.CV]. URL: https://arxiv.org/abs/2405.14458 (cit. on p. 13).

[35] X. Wang, M. Wang, H. Guo, J. Li, X. Wang, and Y. Zhang. "License plate recognition system for complex scenarios based on improved YOLOv5s and LPRNet". In: *Scientific Reports* 15 (Oct. 2025). DOI: 10.1038/s41598-025-18311-4 (cit. on p. 15).

[36] E. Wu. *Faster YOLOv5 inference with TensorRT: Run YOLOv5 at 27 FPS on Jetson Nano*. https://www.seeedstudio.com/blog/2022/08/23/faster-inference-with-tensorrt-on-nvidia-jetson-run-yolov5-at-27-fps-on-jetson-nano/. Accessed: 2026-01-20. 2022 (cit. on p. 15).

[37] A. Zakaria. *European License Plates Dataset*. https://www.kaggle.com/datasets/abdelhamidzakaria/european-license-plates-dataset. Accessed: 2026-07-23 (cit. on p. 22).

[38] S. Zherzdev and A. Gruzdev. *LPRNet: License Plate Recognition via Deep Neural Networks*. 2018. arXiv: 1806.10447 [cs.CV]. URL: https://arxiv.org/abs/1806.10447 (cit. on pp. 14, 17).